

# ENTROPY REGULARIZING ACTIVATION: BOOSTING CONTINUOUS CONTROL, LARGE LANGUAGE MODELS, AND IMAGE CLASSIFICATION WITH ACTIVATION AS ENTROPY CONSTRAINTS

Zilin Kang<sup>1,2\*</sup> Chonghua Liao<sup>3\*</sup> Tingqiang Xu<sup>3\*</sup> Huazhe Xu<sup>1,3,4</sup>

<sup>1</sup>Shanghai Qi Zhi Institute

<sup>2</sup>Department of Computer Science and Technology, Tsinghua University

<sup>3</sup>Institute for Interdisciplinary Information Sciences, Tsinghua University

<sup>4</sup>Shanghai Artificial Intelligence Laboratory

{kz122, lch22, xtq23}@mails.tsinghua.edu.cn

## ABSTRACT

We propose ERA, a new paradigm that constrains the sampling entropy above given thresholds by applying specially designed activations to the outputs of models. Our approach demonstrates broad effectiveness across different domains: 1) for large language models (LLMs), boosting the AIME 2025 score for Qwen2.5-Math-7B by 37.4%; 2) for continuous control reinforcement learning agents, improving performance by more than 30% over strong baselines such as SAC on the challenging HumanoidBench; 3) for image classification, enhancing ImageNet top-1 accuracy by 0.69% for ResNet-50. These gains are achieved with a computational overhead of less than 7%. Our work validates output activation as a powerful tool for entropy control, opening a new direction for designing simpler and more robust algorithms. Code available at: <https://nothingbutbut.github.io/era>

## 1 INTRODUCTION

Decision-making problems represent a broad class of challenges, from robotic control to Large Language Models alignment (Sutton et al., 1998; Ouyang et al., 2022; Kober et al., 2013; Yuan et al., 2025). In these settings, encouraging exploration and maintaining policy stochasticity, often quantified by entropy, is critical (Ziebart et al., 2008; Schulman et al., 2017b). In reinforcement learning, the maximum entropy paradigm, exemplified by algorithms like Soft Actor-Critic (SAC) (Haarnoja et al., 2018), has become a prevailing approach in control tasks. However, these methods, which add an entropy bonus directly to the training objective, inevitably alter the optimization landscape and can interfere with the optimization of the primary objective.

The challenge becomes even more pronounced in LLM alignment. Policy gradient methods (Sutton et al., 1999) such as GRPO (Shao et al., 2024) frequently suffer from entropy collapse (Cui et al., 2025b), leading to reduced diversity and performance degradation. Directly incorporating entropy bonuses has been shown to be unstable or ineffective in this setting (Cui et al., 2025b). Moreover, prior works have explored methods that avoid direct modification of the loss function, including clip-higher (Yu et al., 2025) and training exclusively on the high-entropy tokens (Wang et al., 2025). While these methods provide useful insights, they remain ad hoc, lack a principled mechanism for entropy regulation, and are narrowly tailored to the LLM domain, limiting their applicability to broader settings such as continuous control and computer vision tasks.

These observations highlight a fundamental gap: existing approaches either distort the primary optimization objective, as in RL algorithms with entropy bonus terms, or provide heuristic, domain-specific fixes with no theoretical guarantees, as in LLM alignment. Therefore, there is a pressing need for a new entropy-constraining paradigm that is universally applicable, non-invasive to the primary objective, and theoretically grounded.

---

\*Equal Contribution

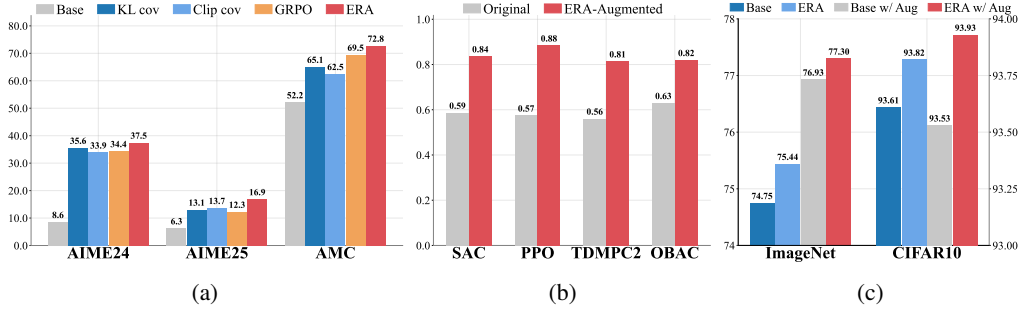


Figure 1: **ERA Boosts Large Language Models, Continuous Control and Image Classification.** (a) **Large Language Models:** ERA consistently enhances the performance of Qwen-2.5-Math-7B on AIME'24, AIME'25 and AMC datasets. (b) **Continuous Control:** ERA significantly improves multiple popular RL algorithms, including SAC, PPO, TD-MPC2 and OBAC. (c) **Image Classification:** ERA consistently boosts the performance of ResNet-50 on ImageNet and CIFAR-10 datasets.

In this work, we introduce **Entropy Regularizing Activation (ERA)**, a novel paradigm for entropy-constrained training. The key insight of ERA lies in imposing an entropy constraint through a class of well-designed activation functions applied to the model's final output. This approach completely decouples the optimization of the primary objective from the entropy constraint, allowing the loss function to focus solely on its original goal (e.g., maximizing rewards). We show that ERA not only provides provable entropy guarantees in theory, but in practice, it functions as a non-invasive module that can be seamlessly integrated with existing algorithms.

The generality and effectiveness of this paradigm are validated across diverse domains, including continuous control, image classification, and large language models. For example, on the DeepMind Control Suite (Tassa et al., 2018), ERA improves the performance of SAC on high-dimensional tasks like Humanoid and Dog by over 25%. Its versatility is also demonstrated in image classification, a domain where preventing model overconfidence via regularization is critical. Our approach complements established methods, boosting performance on top of strong data augmentation and label smoothing (Szegedy et al., 2016). In LLM RL, ERA enables a GRPO-trained Qwen-2.5-Math-7B (Yang et al., 2024b) to achieve a remarkable improvement of 9.0% and 37.4% on the AIME-24 and AIME-25 benchmarks, respectively.

Our main contributions are summarized as follows:

- We introduce **ERA, a novel entropy constraint paradigm** based on activation functions, and establish a theoretical framework with provable entropy guarantees.
- We design effective instantiations of ERA for both continuous (control) and discrete (image classification) domains. For large language models, we propose a **specialized, adaptive variant of ERA that addresses the unique challenges** within this domain.
- Our experiments of these instantiations **demonstrate significant performance improvements** over strong baselines across domains, and reveal their properties such as parameter sensitivity.

## 2 RELATED WORK

**Policy learning in control.** Entropy maximization is a crucial aspect of RL, significantly enhancing exploration and robustness (Ziebart, 2010; Haarnoja et al., 2017). Prior work has explored various methods to incorporate entropy maximization into RL algorithms (O'Donoghue et al., 2016; Nachum et al., 2017; Haarnoja et al., 2017). PPO (Schulman et al., 2017a) introduced an entropy bonus in its clipped surrogate objective. SAC (Haarnoja et al., 2018) later employed a maximum-entropy objective with a dynamically adjusted temperature parameter, but this can lead to instability. More recent approaches have introduced alternative methodologies for implementing maximum entropy RL (Chao et al., 2024; Choe & Kim, 2024), while others have shifted the optimization focus directly to state entropy (Zhong et al., 2024). All these methods, while effective, modify the original cumulative reward objective by introducing the entropy term, which can lead to suboptimal performance. Our approach addresses this issue by maintaining the original objective, ensuring more reliable performance.

**RL for LLMs.** Recent breakthroughs in LLM reasoning, such as OpenAI-o1 (Jaech et al., 2024), DeepSeek-R1 (Guo et al., 2025), and Kimi-k1.5 (Team et al., 2025), have redirected attention from chain-of-thought prompting (Wei et al., 2022) and supervised fine-tuning (Li et al., 2024a; Yeo et al., 2025) toward RL. Within this paradigm, policy entropy collapse emerges as a fundamental obstacle: the decay of exploratory behavior often leads to performance plateaus. A prevalent approach is reward shaping (Cheng et al., 2025), which augments the reward or advantage with an entropy bonus to maintain a viable exploration–exploitation trade-off. Complementary strategies, including loss re-weighting (Wang et al., 2025; Cui et al., 2025b) and clip-higher regularization (Yu et al., 2025), mitigate the risk of entropy collapse. Unlike these approaches, our method is a general and concise paradigm, universally applicable across domains and endowed with rigorous theoretical guarantees.

### 3 PRELIMINARIES

**Policy optimization.** Policy gradient (PG) methods optimize  $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^T \gamma^t R(s_t, a_t) \right]$  via gradient ascent. For large language model (LLM) alignment, Proximal Policy Optimization (PPO) (Schulman et al., 2017b) is commonly used. The GRPO variant estimates the advantage  $A(y)$  for a generated response  $y$  from a set of  $K$  samples as:

$$A(y) = \frac{r(y) - \text{mean}(r(y^{1:K}))}{\text{std}(r(y^{1:K}))}. \quad (1)$$

The policy is then updated using the clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (2)$$

where  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$  is the probability ratio.

**Policy entropy.** Policy entropy,  $\mathcal{H}(\pi(\cdot|s))$ , measures the policy’s stochasticity. For discrete action spaces, the token-level entropy is given by Eq. 3. For continuous policies, there are several common ways to ensure actions remain within a bounded space. A popular method is to use a squashed Gaussian policy, which outputs a bounded action  $a = \tanh(u)$  by sampling  $u$  from a Gaussian distribution  $\pi_\theta(\cdot|s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$  parameterized by the policy network. The entropy of this policy is given by Eq. 4. Alternatively, another common approach is to directly sample actions from a Truncated Gaussian distribution  $\pi_\theta(\cdot|s) = \text{TN}(\mu_\theta(s), \Sigma_\theta(s), -1, 1)$  over the bounded hypercube  $[-1, 1]^D$ . Assuming the dimensions are independent, its entropy is given by Eq. 5.

$$\mathcal{H}(\pi_\theta) = -\mathbb{E}_{x \sim \rho_\pi, y \sim \pi_\theta(x)} \left[ \frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_\theta(y_t | y_{<t}, x) \right], \quad (3)$$

$$\mathcal{H}(\pi_\theta) = \mathbb{E}_{s \sim \rho_\pi, u \sim \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))} \left[ -\log \mathcal{N}(u | \mu_\theta(s), \Sigma_\theta(s)) + \sum_{i=1}^D \log(1 - \tanh(u_i)^2) \right], \quad (4)$$

$$\mathcal{H}(\pi_\theta) = \mathbb{E}_{s \sim \rho_\pi} \left[ \sum_{i=1}^D \left( \log(\sigma_{\theta,i}(s) Z_i(s) \sqrt{2\pi e}) - \frac{\beta_i(s) \phi(\beta_i(s)) - \alpha_i(s) \phi(\alpha_i(s))}{2Z_i(s)} \right) \right] \quad (5)$$

where for the truncated Gaussian entropy in Eq. 5,  $\phi$  and  $\Phi$  are the PDF and CDF of the standard normal distribution, respectively. We define the standardized bounds  $\alpha_i(s) = (-1 - \mu_{\theta,i}(s))/\sigma_{\theta,i}(s)$ ,  $\beta_i(s) = (1 - \mu_{\theta,i}(s))/\sigma_{\theta,i}(s)$ , and the normalization constant  $Z_i(s) = \Phi(\beta_i(s)) - \Phi(\alpha_i(s))$ .

**Maximum entropy reinforcement learning.** Building upon policy entropy, the maximum entropy RL framework aims to maximize the standard reward objective subject to a minimum entropy constraint  $\mathcal{H}_0$ :

$$\max_{\theta} J(\pi_\theta) \quad \text{s.t.} \quad \mathbb{E}_{s \sim \rho_\pi} [\mathcal{H}(\pi_\theta(\cdot|s))] \geq \mathcal{H}_0. \quad (6)$$

Practical algorithms like Soft Actor-Critic (SAC) (Haarnoja et al., 2018) solve the Lagrangian dual of this problem. SAC is an off-policy actor-critic algorithm that updates a soft Q-function  $Q_\phi$  and a policy  $\pi_\theta$ . The Q-function is updated by minimizing the soft Bellman residual  $J_Q(\phi)$ :

$$J_Q(\phi) = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[ \frac{1}{2} (Q_\phi(s_t, a_t) - y)^2 \right] \quad (7)$$

$$y = R(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi_{\theta}(\cdot|s_{t+1})} [Q_{\phi'}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1}|s_{t+1})] \quad (8)$$

with the target  $y$  computed using a target Q-network  $Q_{\phi'}$ . The target network parameters  $\phi'$  are updated via an exponential moving average (EMA):  $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$ .

$$J_{\pi}(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_{\theta}} [Q_{\phi}(s_t, a_t) - \alpha \log \pi_{\theta}(a_t|s_t)]. \quad (9)$$

The policy is then updated by maximizing the objective in Eq. 9.

## 4 THE ENTROPY REGULARIZING ACTIVATION

### 4.1 THE CORE IDEA: ENTROPY CONSTRAINT VIA OUTPUT ACTIVATION

The core of Entropy Regularizing Activation is to enforce maximum entropy reinforcement learning on the policy, not through a loss penalty, but via integrating the constraint into the network's architecture via a special activation function.

Let a parameterized policy  $f_{\theta}(s)$  produce distribution parameters  $z = f_{\theta}(s)$ , where  $z$  belongs to a parameter space  $\mathcal{Z}$ . The policy corresponding to these parameters is  $\pi_z(\cdot|s)$ . We introduce an activation function  $g : \mathcal{Z} \rightarrow \mathcal{Z}$ , which transforms the initial parameters  $z$  to a new set  $z' = g(z)$ . The final policy, which we denote as  $\pi_{\theta}$ , is thus given by  $\pi_{\theta}(\cdot|s) = \pi_{g(f_{\theta}(s))}(\cdot|s)$ . The function  $g(\cdot)$  is designed to ensure that the policy  $\pi_{\theta}$  satisfies a constraint on its expected entropy, for a given target entropy  $\mathcal{H}_0$ :

$$\mathbb{E}_{s \sim \rho_{\pi}} [\mathcal{H}_{\pi_{\theta}(\cdot|s)}] \geq \mathcal{H}_0$$

This formulation enables the policy to satisfy the expected entropy condition while leaving the training objective for  $\theta$  free of an explicit entropy term, as shown in Eq. 6. This approach effectively mitigates gradient conflicts between the task objective and the entropy maximization objective, allowing the optimization to focus on the primary objective.

### 4.2 INSTANTIATIONS FOR CONTINUOUS AND DISCRETE SPACES

To ground the general framework presented in section 4.1, we now instantiate the entropy regularizing activation  $g(\cdot)$  for two canonical policy classes: policies based on a bounded Gaussian distribution, such as the Tanh-squashed Gaussian (Haarnoja et al., 2018) or the clipped Gaussian (Fujimoto et al., 2018), commonly used in continuous control; and the softmax policy prevalent in discrete spaces.

#### 4.2.1 CONTINUOUS CONTROL WITH BOUNDED GAUSSIAN POLICIES

In continuous control, policies often sample actions from a Gaussian distribution and then apply a bounding function (e.g., a tanh squash or clipping) to ensure outputs lie within a valid range. This bounding operation complicates direct entropy maximization, as it introduces a state-dependent bias term. Prior methods typically address this by adding an entropy bonus to the learning objective. Our insight is that the entropy of the final bounded policy,  $\mathcal{H}_{\pi}$ , can be seen as the entropy of the original unbounded Gaussian,  $\mathcal{H}_{\text{Gaussian}}$ , minus a non-negative bias term introduced by the bounding operation, i.e.,  $\mathcal{H}_{\pi} = \mathcal{H}_{\text{Gaussian}} - \mathbb{E}[\text{bias}]$ . Consequently, a minimum entropy constraint on the final policy can be satisfied by constraining the underlying Gaussian's entropy to a corresponding, higher value. This is achieved by adjusting the Gaussian's standard deviation,  $\sigma$ . The entropy of a  $D$ -dimensional Gaussian with a diagonal covariance matrix is:

$$\mathcal{H}_{\text{Gaussian}}(s) = \frac{1}{2} \sum_{i=1}^D \log(2\pi e \sigma_i(s)^2) \quad (10)$$

To maintain training stability, the standard deviation must also be kept within a predefined range  $[\sigma_{\min}, \sigma_{\max}]$ . Our activation function  $g(\cdot)$  simultaneously satisfies both constraints. Given network outputs for the mean  $\mu$  and a pre-activation standard deviation  $\hat{\sigma}$ , the function  $g(\mu, \hat{\sigma})$  produces the final parameters  $(\mu', \sigma')$  where:

$$\mu' = \mu, \quad \sigma' = \exp \left[ \max \left( \log \sigma_{\max} + (\mathcal{H}'_0 - D \log \sqrt{2\pi e} - D \log \sigma_{\max}) \frac{e^{\hat{\sigma}_i}}{\sum_{j=1}^D e^{\hat{\sigma}_j}}, \log \sigma_{\min} \right) \right] \quad (11)$$

Here,  $\mathcal{H}'_0$  is the target entropy for the final policy  $\mathcal{H}_0$  plus a compensation parameter  $\delta \geq 0$  to account for the bounding bias, which can either be a constant or automatically tuned by learning with the loss in Eq. 12.

$$\mathcal{L}(\hat{\delta}) = \mathbb{E}_{s \sim \mathcal{D}} \left[ \hat{\delta} (\mathcal{H}[\pi(\cdot|s)] - \mathcal{H}_0) \right] \quad (12)$$

We refer the reader to Appendix A.1 for implementation details and Appendix B.1 for a proof of the entropy bound.

By satisfying the entropy constraint architecturally, our method obviates the need for an explicit entropy term in the objective function. Hence, target of the critic and the actor loss of SAC in Eq. 8 and Eq. 9 can be simplified to the form in Eq. 13 and Eq. 14

$$y = R(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\theta(\cdot|s_{t+1})} \left[ Q_{\phi'}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1}|s_{t+1}) \right] \quad (13)$$

$$J_\pi(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\theta} \left[ Q_\phi(s_t, a_t) - \alpha \log \pi_\theta(a_t|s_t) \right] \quad (14)$$

#### 4.2.2 DISCRETE CLASSIFICATION WITH SOFTMAX POLICIES

In discrete classification, regularizing the predictive entropy is crucial for preventing the overconfidence that leads to overfitting. ERA provides architectural regularization by enforcing a minimum entropy level, analogous to how techniques like label smoothing improve generalization by smoothing the output distribution. For a softmax policy, we enforce this constraint by transforming the pre-activation logits  $z$  into  $z'$  such that the resulting policy’s entropy is at least  $\mathcal{H}_0$ :

$$z' = h^{-1} \left[ \max \left( \frac{\log \tau}{\tau} + \left( C_{\mathcal{H}_0} - n \frac{\log \tau}{\tau} \right) \frac{1}{D-1} \left( 1 - \frac{e^{z_i}}{\sum_{j=1}^D e^{z_j}} \right), 0 \right) \right] \quad (15)$$

Here,  $h^{-1}$  denotes the inverse of  $\ln(-x \ln x)$  on  $[0, \frac{1}{e}]$ , approximated by  $\hat{h}^{-1}(x) = -\frac{1}{4} - \sqrt{2(-1 - \ln(x))} + \frac{3}{4} \ln x$ . We also define  $C_{\mathcal{H}_0} = \exp(\mathcal{H}_0 - 1)$ , where  $\tau \geq e$  is a fixed hyperparameter (e.g.,  $\tau = 4$ ). A formal proof is provided in Appendix B.2.

In contrast to label smoothing, which applies a fixed and uniform regularization, ERA offers greater flexibility. It allows the model to learn a structured, input-dependent uncertainty distribution, tailoring the regularization to each sample and thus offering greater expressive capacity and potential for improved performance.

#### 4.3 INSTANTIATIONS FOR RL IN LARGE LANGUAGE MODELS

In reinforcement learning for LLMs, each token is treated as a discrete action, with the policy defined by a canonical softmax distribution. Prior approaches to addressing entropy collapse in LLMs—such as the traditional entropy bonus, clip-higher, KL-Cov, and Clip-Cov—do not provide a provable entropy lower bound, and are incompatible with the on-policy setting. In contrast, our method introduces ERA, a simple and non-invasive activation function that offers a theoretical guarantee of a minimum entropy level, effectively resolving entropy collapse in on-policy reinforcement learning.

In contrast to standard RL settings, the action space is extremely large. In the previous ERA instantiation, each token has a lower entropy bound. However, due to the intrinsic structure of natural language, most tokens are nearly deterministic; therefore, directly enforcing high entropy across all tokens is impractical: it will lead to unintended tokens and can corrupt the entire response. Furthermore, modifying the internal structure of the model also introduces instability in different training environments, leading to unpredictable behavior.

To address these challenges, we propose a new instantiation of ERA that is applied *after* the sampling process. Specifically, responses are first generated using the original model output  $z$ , and the advantages are computed following the GRPO rule. Then, during model updates, the probabilities of the sampled tokens are reinterpreted as  $z'$ , obtained by applying our entropy-regularized activation. This design leaves the sampling policy unchanged while still ensuring effective entropy regularization.

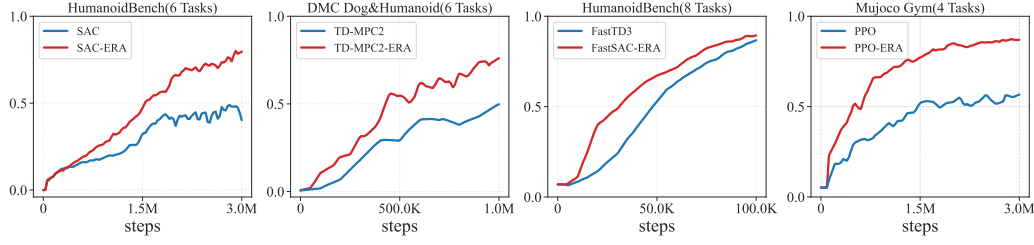


Figure 2: **Main Results of ERA in Continuous Control.** Aggregate normalized performance on HumanoidBench (6 tasks, with SAC), DMC (Humanoid & Dog) (6 tasks, with TD-MPC2), HumanoidBench (8 tasks, with FastSAC) and Mujoco Gym (4 tasks, with PPO). ERA consistently accelerates learning and achieves superior asymptotic performance.

Formally, when updating model parameters, we apply an activation layer to the logits  $z$  to obtain a transformed set  $z'$ , defined as:

$$z' = \begin{cases} kz & H_{\text{resp}} < \omega_{\text{low}}, A_t > 0, \\ z & \omega_{\text{low}} \leq H_{\text{resp}} \leq \omega_{\text{high}}, A_t < 0 \text{ or } A_t > 0, \\ \frac{1}{k}z & H_{\text{resp}} > \omega_{\text{high}}, A_t > 0, \end{cases} \quad (16)$$

where  $k > 1$ , and  $\omega_{\text{low}}, \omega_{\text{high}}$  are algorithm-specific constants. Here,  $A_t$  denotes the advantage of the token, and  $H_{\text{resp}}$  is the average entropy of the top 20% of tokens with the highest entropy in the response. To balance the gradient between modified tokens and unmodified tokens (details are shown in Appendix B.3), we add another scaling factor on the advantages of modified tokens:

$$A'_t = \begin{cases} \frac{1}{k}A_t & H_{\text{resp}} < \omega_{\text{low}}, A_t > 0, \\ A_t & \omega_{\text{low}} \leq H_{\text{resp}} \leq \omega_{\text{high}}, A_t < 0 \text{ or } A_t > 0, \\ kA_t & H_{\text{resp}} > \omega_{\text{high}}, A_t > 0, \end{cases} \quad (17)$$

The on-policy GRPO objective becomes:

$$J(\theta) = \mathbb{E}_t[\mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)} \log \pi'_\theta(a_t|s_t) A'_t] \quad (18)$$

where  $\pi_\theta$  is the original policy from  $z$  (representing that the inference still follows the original policy), and  $\pi'_\theta$  is the ERA-adjusted policy from  $z'$  (representing that the model update relies on the new policy). Intuitively, this activation layer adjusts all positively advantaged responses: when entropy is too low, it sharpens the probability distribution; when entropy is too high, it flattens it. Unlike our instantiation for control tasks, increasing policy entropy here requires *sharpening* the distribution. The rationale is that sampling has already occurred, and by treating the samples as if they were drawn from a sharpened policy, the model perceives itself as overexploiting, thus encouraging additional exploration. The choice of the top 20% tokens is based on the fact that, in natural language, these tokens are considered forking tokens, whose entropy is the target of regularization, and the remaining tokens are allowed to have almost zero entropy (Wang et al., 2025).

We show that, under reasonable assumptions, this ERA instantiation ensures that the policy entropy remains above a fixed constant  $\mathcal{H}_0$ . We refer the reader to Appendix B.3 for a formal proof.

## 5 RESULTS AND ANALYSIS

### 5.1 EXPERIMENTS ON CONTINUOUS CONTROL

We conduct extensive experiments to validate the effectiveness of ERA in continuous control tasks. We demonstrate the broad applicability and performance gains by integrating ERA into five distinct algorithms—SAC, OBAC (Luo et al., 2024), TD-MPC2, PPO, and FastSAC (Seo et al., 2025). The evaluation is performed on a wide range of challenging benchmarks, including the DeepMind Control Suite (Humanoid & Dog), HumanoidBench (Sferrazza et al., 2024), and MuJoCo Gym (Todorov et al., 2012). Implementation details, environment specifics, and hyperparameter settings are available in Appendix A.1. Comprehensive results for all tasks can be found in the Appendix C.



**Main results.** We present our main results in continuous control in Figure 2. Integrating ERA consistently yields significant improvements in both sample efficiency and final performance across diverse algorithms and benchmarks.

**ERA consistently improves performance across various entropy targets.** We evaluate the performance of SAC and SAC-ERA under varying entropy targets. The results in Figure 3a, tested on four DMC tasks (*dog-run*, *dog-trot*, *humanoid-run*, *humanoid-walk*) with 5 seeds on each environment, show that SAC-ERA consistently outperforms original SAC across the entire tested spectrum of entropy values. By bypassing the entropy constraint within the learning objective, ERA allows the policy to focus more directly on reward maximization. While simply removing the entropy term from SAC can also avoid this constraint, its performance is inferior to the ERA-enhanced version due to insufficient exploration. This consistent outperformance suggests that ERA can achieve strong results without precise tuning of the entropy hyperparameter, offering a significant practical advantage.

## 5.2 EXPERIMENTS ON IMAGE CLASSIFICATION

Table 1: Top-1 and Top-5 accuracy (%) on ImageNet and CIFAR-10. We compare ERA against the original ResNet-50 baseline.  $\Delta$  denotes the absolute improvement of ERA. All models are trained for 200 epochs.

| Dataset  | Method   | Without Data Augmentation          |              |                                    |              | With Data Augmentation             |              |                                    |              |
|----------|----------|------------------------------------|--------------|------------------------------------|--------------|------------------------------------|--------------|------------------------------------|--------------|
|          |          | Top-1 Acc.                         | $\Delta$     | Top-5 Acc.                         | $\Delta$     | Top-1 Acc.                         | $\Delta$     | Top-5 Acc.                         | $\Delta$     |
| ImageNet | Original | 74.75 $\pm$ 0.38                   | -            | 92.04 $\pm$ 0.23                   | -            | 76.93 $\pm$ 0.36                   | -            | 93.37 $\pm$ 0.21                   | -            |
|          | ERA      | <b>75.44 <math>\pm</math> 0.37</b> | <b>+0.69</b> | <b>92.15 <math>\pm</math> 0.23</b> | <b>+0.11</b> | <b>77.30 <math>\pm</math> 0.36</b> | <b>+0.37</b> | <b>93.39 <math>\pm</math> 0.21</b> | <b>+0.02</b> |
| CIFAR-10 | Original | 93.61 $\pm$ 0.14                   | -            | 99.69 $\pm$ 0.08                   | -            | 93.53 $\pm$ 0.03                   | -            | 99.84 $\pm$ 0.02                   | -            |
|          | ERA      | <b>93.82 <math>\pm</math> 0.08</b> | <b>+0.21</b> | <b>99.82 <math>\pm</math> 0.03</b> | <b>+0.13</b> | <b>93.93 <math>\pm</math> 0.12</b> | <b>+0.4</b>  | <b>99.86 <math>\pm</math> 0.01</b> | <b>+0.02</b> |

We evaluate our method on the ImageNet (Russakovsky et al., 2015) and CIFAR-10 datasets (Krizhevsky et al., 2009). Our implementation utilizes the ResNet-50 architecture from the PyTorch Image Models (timm) library (Wightman, 2019). To ensure a fair comparison, both our method and the baseline were trained for 200 epochs, with all other hyperparameters held constant. Notably, we retain key default settings from timm for all experiments, including a label smoothing factor of 0.1. This demonstrate ERA’s complementarity with existing regularizations.

**Main results.** Table 1 summarizes the primary classification results, comparing ERA against the standard ResNet-50 baseline. For these results, we use a minimal entropy of 1.2 for ImageNet and 0.6 for CIFAR-10. The comparison is conducted under two settings: with and without the standard data augmentation provided by the timm library. The results show that ERA consistently outperforms the baseline across both datasets and settings.

**Ablation study on minimal entropy.** We study our method’s robustness to the minimal entropy hyperparameter on ImageNet and CIFAR-10, using checkpoints from the 100th and 200th epochs, respectively, for efficiency. As shown in Figure 3b, our method exhibits low sensitivity to this parameter. Rather than fine-tuning for peak performance, our intent is to show that competitive accuracy is maintained across a reasonable range of values. This demonstrates strong performance is achievable without extensive tuning.

## 5.3 RESULTS AND ANALYSIS ON LARGE LANGUAGE MODELS

We first present the results of ERA in §5.3.1 Main Results and §5.3.2 Extension to More Models and Algorithms. We then use §5.3.3 Analysis on Entropy and Reasoning Capacity Boundary and §5.3.4 Out-of-Distribution Generalization to illustrate the role of encouraging exploration. Additional ablation studies on method design are provided in the Appendix C.3.

### 5.3.1 MAIN RESULTS

We evaluate ERA on Qwen2.5-Math-7B, trained with the DAPO-Math-17K (Yu et al., 2025) dataset using codebase adopted from verl (Sheng et al., 2025). To improve training stability and ensure well-controlled entropy decay, we adopt a two-stage training strategy. In the first stage, we set

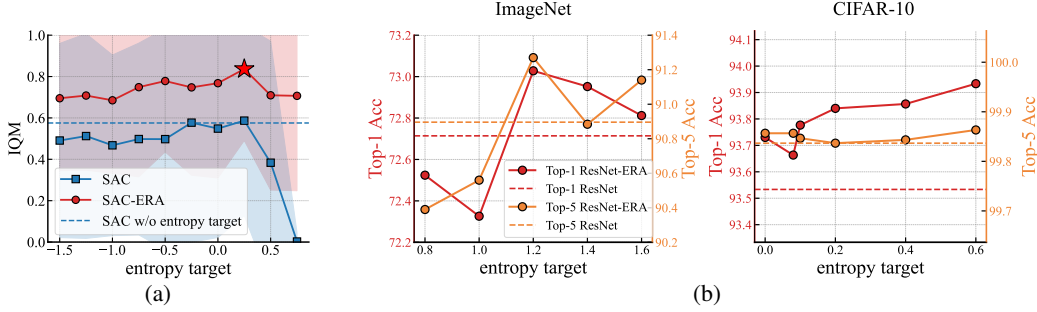


Figure 3: **Sensitivity of ERA to the Minimum Entropy.** (a) **1M Steps Performance on DMC Tasks.** Comparison between SAC-ERA and the baseline SAC on Humanoid and Dogs environments under various minimum entropy constraints. Our method achieves superior performance across all settings. (b) **Accuracy on ImageNet and CIFAR-10.** ResNet-ERA maintains stable Top-1 and Top-5 accuracy across a range of minimum entropy values, indicating its robustness to the choice of this hyperparameter.

$\omega_{\text{low}} = 0.45$ ,  $\omega_{\text{high}} = 3.0$ , and  $k = 2$ , and train for 600 steps. In the second stage, we continue training for 500 steps with a relaxed entropy bound, setting  $\omega_{\text{low}} = 0.2$ ,  $\omega_{\text{high}} = +\infty$ , and keeping  $k = 2$ .

We then evaluate the resulting model on six standard mathematical reasoning tasks: AIME’24, AIME’25, AMC’23 (Li et al., 2024b), MATH500 (Hendrycks et al., 2021), Minerva (Lewkowycz et al., 2022), and OlympiadBench (He et al., 2024). Table 2 presents comparisons against base models, classical RL methods, and recent entropy-control approaches. AIME’24, AIME’25, and AMC’23 are conducted with a decoding temperature of 0.7, and reported as the average accuracy over 16 sampled responses. MATH500, Minerva, and OlympiadBench are conducted with greedy sampling. The evaluation process is sampled on the original policy  $z$  (before ERA). Full implementation details and hyperparameter settings are provided in Appendix A.3. The results show that ERA consistently achieves the best results on most of the benchmarks. Notably, it outperforms strong entropy-based baselines such as KL-Cov and Clip-Cov by significant margins.

Table 2: Main results (%) on five competition-level reasoning benchmarks based on Qwen2.5-Math-7B. For AIME and AMC, the results are avg. @ 16. The best results on each benchmark are highlighted in **bold**.

| Model                                          | AIME24 $\uparrow$ | AIME25 $\uparrow$ | AMC $\uparrow$ | MATH500 $\uparrow$ | Minerva $\uparrow$ | Olympiad $\uparrow$ | Avg. $\uparrow$ |
|------------------------------------------------|-------------------|-------------------|----------------|--------------------|--------------------|---------------------|-----------------|
| <b>Base Models</b>                             |                   |                   |                |                    |                    |                     |                 |
| Qwen2.5-Math Yang et al. (2024a)               | 8.6               | 6.3               | 52.2           | 50.8               | 12.1               | 17.2                | 24.5            |
| Qwen2.5-Math-Instruct Yang et al. (2024a)      | 13.3              | 10.0              | 57.1           | 81.0               | 32.7               | 38.8                | 38.8            |
| <b>Classical Methods</b>                       |                   |                   |                |                    |                    |                     |                 |
| SimpleRL-Zero Zeng et al. (2025)               | 26.7              | 9.3               | 60.0           | 74.6               | 27.6               | 35.8                | 39.0            |
| OpenReasoner-Zero Hu et al. (2025)             | 15.4              | 13.4              | 56.5           | 81.0               | 32.7               | 43.2                | 40.4            |
| PRIME-Zero Cui et al. (2025a)                  | 18.9              | 11.7              | 57.7           | 79.0               | 36.4               | 40.6                | 40.7            |
| Oat-Zero Liu et al. (2025)                     | 28.8              | 10.8              | 65.2           | 79.6               | 34.2               | 39.9                | 43.1            |
| <b>Entropy Control Methods</b>                 |                   |                   |                |                    |                    |                     |                 |
| GRPO w/ 20% Forking Tokens (Wang et al., 2025) | 29.0              | <b>17.7</b>       | 63.6           | 81.8               | 39.7               | 44.6                | 46.1            |
| KL-Cov (Cui et al., 2025b)                     | 35.6              | 13.1              | 65.1           | 81.0               | 40.4               | 44.1                | 46.6            |
| Clip-Cov (Cui et al., 2025b)                   | 33.9              | 13.7              | 62.5           | 78.4               | 35.6               | 40.3                | 44.1            |
| GRPO (Shao et al., 2024)                       | 34.4              | 12.3              | 69.5           | 80.6               | 36.8               | 40.6                | 45.7            |
| ERA                                            | <b>37.5</b>       | 16.9              | <b>72.8</b>    | <b>84.6</b>        | <b>42.6</b>        | <b>46.5</b>         | <b>50.2</b>     |
| $\Delta$ ( $\uparrow$ )                        | <b>+9.0%</b>      | <b>+37.4%</b>     | <b>+4.7%</b>   | <b>+5.0%</b>       | <b>+15.8%</b>      | <b>+14.5%</b>       | <b>+9.8%</b>    |

### 5.3.2 EXTENSION TO MORE MODELS AND ALGORITHMS

To demonstrate ERA’s effectiveness across different model sizes and algorithms, we extend it to the weaker Qwen2.5-Math-1.5B model and also apply ERA to other algorithms such as GSPO (Zheng et al., 2025) on Qwen2.5-Math-7B, showing that ERA is a generic approach not tied to any specific model or algorithm. As reported in Table 3, ERA yields significant gains on both the smaller model and GSPO. For instance, on Qwen2.5-Math-1.5B it achieves an average improvement of 14.1%.

### 5.3.3 ANALYSIS ON ENTROPY AND REASONING CAPACITY BOUNDARY

To better understand the effect of our approach on exploration and reasoning, we examine both the entropy dynamics of the learned policies and their downstream reasoning performance. Figure 4



Table 3: Accuracy (%) results of different LLMs and different algorithms across six benchmarks. The best results in each box are highlighted in **bold**.

| Method                                | AIME24 $\uparrow$ | AIME25 $\uparrow$ | AMC $\uparrow$ | MATH500 $\uparrow$ | Minerva $\uparrow$ | Olympiad $\uparrow$ | Avg. $\uparrow$ |
|---------------------------------------|-------------------|-------------------|----------------|--------------------|--------------------|---------------------|-----------------|
| Qwen2.5-Math-1.5B Yang et al. (2024a) |                   |                   |                |                    |                    |                     |                 |
| CoT                                   | 4.3               | 2.3               | 26.4           | 59.0               | 24.3               | 27.6                | 24.0            |
| GRPO                                  | 11.1              | 6.0               | 40.2           | 66.4               | 25.0               | 30.1                | 29.8            |
| ERA                                   | <b>12.1</b>       | <b>6.8</b>        | <b>49.5</b>    | <b>70.6</b>        | <b>30.5</b>        | <b>34.7</b>         | <b>34.0</b>     |
| $\Delta$ ( $\uparrow$ )               | +9.0%             | +13.3%            | +23.1%         | +6.3%              | +22.0%             | +15.3%              | +14.1%          |
| Qwen2.5-Math-7B Yang et al. (2024a)   |                   |                   |                |                    |                    |                     |                 |
| CoT                                   | 8.6               | 6.3               | 52.2           | 50.8               | 12.1               | 17.2                | 24.5            |
| GSPO                                  | 29.8              | 13.7              | 61.2           | <b>85.1</b>        | 37.1               | 35.1                | 43.7            |
| GSPO + ERA                            | <b>33.3</b>       | <b>15.2</b>       | <b>63.8</b>    | 84.3               | <b>40.8</b>        | <b>42.7</b>         | <b>46.7</b>     |
| $\Delta$ ( $\uparrow$ )               | +11.7%            | +10.9%            | +4.2%          | -0.9%              | +10.0%             | +21.7%              | +6.9%           |

compares the entropy trajectories of our method (first stage) with the GRPO baseline. While GRPO suffers from entropy collapse, our method maintains a stable entropy level throughout training. This stability indicates the existence of a *non-trivial entropy lower bound*, as we desired by the definition of ERA, which prevents premature policy concentration and preserves the model’s ability to explore diverse reasoning paths.

The presence of this entropy floor aligns with improved reasoning performance. As shown in Figure 4, ERA achieves consistently higher pass@ $k$  scores on AIME’24 and AIME’25 compared to GRPO. This demonstrates that avoiding entropy collapse is not merely a statistical artifact but translates directly into stronger reasoning capacity. In particular, maintaining sufficient entropy ensures the model retains multiple candidate reasoning trajectories, thereby improving the likelihood of successful solutions under pass@ $k$  evaluation.

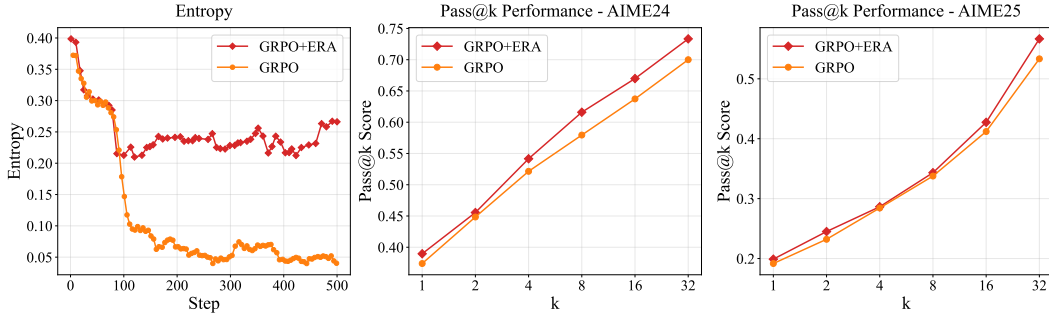


Figure 4: **Entropy comparison and pass@ $k$  results for GRPO with ERA (ours) versus GRPO alone.** The entropy curves demonstrate that ERA mitigates entropy collapse and establishes a clear lower bound. The pass@ $k$  results further indicate that ERA enhances exploration and strengthens the model’s reasoning ability.

#### 5.3.4 OUT-OF-DISTRIBUTION GENERALIZATION

Models trained in a specific domain often struggle when applied to other domains (Yuan et al., 2023; Wang et al., 2024a). Since ERA uses entropy constraints to encourage exploration, we hope it can learn *more general skills*. Therefore we want to see if ERA will also do better on out-of-distribution (OOD) data than standard GRPO. To test this, we evaluate ERA on three hard OOD benchmarks: ARC-C (Clark et al., 2018), GPQA-Diamond (Rein et al., 2024), and MMLU-Pro (Wang et al., 2024b). As shown in Figure 5, ERA outperforms GRPO by 16.9% on average. This confirms our hypothesis that ERA can also enable models to learn more generalizable abilities.

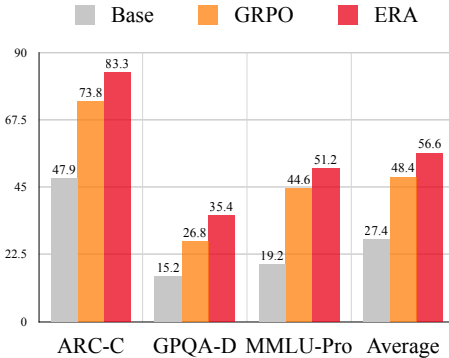


Figure 5: Results on three OOD benchmarks (Qwen2.5-Math-7B).

## 6 CONCLUSIONS

In this work, we introduced ERA, a novel entropy-constrained paradigm built upon the unique principle of treating output activations as a direct medium for entropy regularization. Our theoretical analysis is substantiated by strong empirical results across diverse and challenging domains. In these settings, ERA consistently surpasses prominent baselines without incurring significant computational overhead. Ultimately, this work offers a new perspective on entropy regularization for both supervised and unsupervised decision-making, opening a promising research avenue for developing more robust and efficient learning agents.

## REPRODUCIBILITY STATEMENT

We are strongly committed to the reproducibility of our work. To this end, we provide detailed derivations and proofs for all theoretical claims in the appendix. The appendix also contains comprehensive experimental details, including hyperparameters, environment setups, and additional results, which are crucial for replicating our findings. Furthermore, the core source code for our proposed method, ERA, instantiated across all domains, is included in the appendix. As our implementations are built upon publicly available codebases and frameworks, we believe the provided key source code is sufficient for a straightforward reproduction of our results. Additionally, a full open-source codebase is available at: <https://nothingbutbut.github.io/era>

## ACKNOWLEDGMENT

This work was supported by the Tsinghua University Initiative Scientific Research Program No. 20257020004. We would also like to express our gratitude to Kaizhe Hu, Ruizhe Shi, and Huanyu Li from the Tsinghua Embodied AI Lab for their invaluable discussions and insightful feedback, which have significantly contributed to this work.

## REFERENCES

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Onur Celik, Zechu Li, Denis Blessing, Ge Li, Daniel Palenicek, Jan Peters, Georgia Chalvatzaki, and Gerhard Neumann. Dime:diffusion-based maximum entropy reinforcement learning, 2025. URL <https://arxiv.org/abs/2502.02316>.
- Chen-Hao Chao, Chien Feng, Wei-Fang Sun, Cheng-Kuang Lee, Simon See, and Chun-Yi Lee. Maximum entropy reinforcement learning via energy-based normalizing flow, 2024. URL <https://arxiv.org/abs/2405.13629>.
- Daixuan Cheng, Shaohan Huang, Xuekai Zhu, Bo Dai, Wayne Xin Zhao, Zhenliang Zhang, and Furu Wei. Reasoning with exploration: An entropy perspective. *arXiv preprint arXiv:2506.14758*, 2025.
- Jean Seong Bjorn Choe and Jong-Kook Kim. Maximum entropy on-policy actor-critic via entropy advantage estimation, 2024. URL <https://arxiv.org/abs/2407.18143>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025a.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025b.

- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods, 2018. URL <https://arxiv.org/abs/1802.09477>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pp. 1352–1361. PMLR, 2017.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model, 2025. URL <https://arxiv.org/abs/2503.24290>.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015. URL <https://arxiv.org/abs/1502.03167>.
- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Zilin Kang, Chenyuan Hu, Yu Luo, Zhecheng Yuan, Ruijie Zheng, and Huazhe Xu. A forget-and-grow strategy for deep reinforcement learning scaling in continuous control, 2025. URL <https://arxiv.org/abs/2507.02712>.
- Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.
- Ilya Kostrikov. JAXRL: Implementations of Reinforcement Learning algorithms in JAX, 10 2021. URL <https://github.com/ikostrikov/jaxrl>.
- Alex Krizhevsky et al. Learning multiple layers of features from tiny images. 2009.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857, 2022.
- Chen Li, Weiqi Wang, Jingcheng Hu, Yixuan Wei, Nanning Zheng, Han Hu, Zheng Zhang, and Houwen Peng. Common 7b language models already possess strong math capabilities. *arXiv preprint arXiv:2403.04706*, 2024a.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. <https://huggingface.co/datasets/Numinamath>, 2024b. Hugging Face repository, 13:9.

- Zhi Li. DRL-code-pytorch. <https://github.com/Lizhi-sjtu/DRL-code-pytorch>, 2022. Accessed: 2025-09-10.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Yu Luo, Tianying Ji, Fuchun Sun, Jianwei Zhang, Huazhe Xu, and Xianyuan Zhan. Offline-boosted actor-critic: Adaptively blending optimal historical behaviors in deep off-policy rl. *arXiv preprint arXiv:2405.18520*, 2024.
- Haitong Ma, Tianyi Chen, Kai Wang, Na Li, and Bo Dai. Efficient online reinforcement learning for diffusion policy, 2025. URL <https://arxiv.org/abs/2502.00361>.
- Ofir Nachum, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. Bridging the gap between value and policy based reinforcement learning. *Advances in neural information processing systems*, 30, 2017.
- Michał Nauman, Michał Bortkiewicz, Piotr Miłoś, Tomasz Trzcinski, Mateusz Ostaszewski, and Marek Cygan. Overestimation, overfitting, and plasticity in actor-critic: the bitter lesson of reinforcement learning, 2024. URL <https://arxiv.org/abs/2403.00514>.
- Brendan O’Donoghue, Remi Munos, Koray Kavukcuoglu, and Volodymyr Mnih. Combining policy gradient and q-learning. *arXiv preprint arXiv:1611.01626*, 2016.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge, 2015. URL <https://arxiv.org/abs/1409.0575>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017a.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017b.
- Younggyo Seo, Carmelo Sferrazza, Haoran Geng, Michal Nauman, Zhao-Heng Yin, and Pieter Abbeel. Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv preprint arXiv:2505.22642*, 2025.
- Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoid-bench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL [https://proceedings.neurips.cc/paper\\_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf).
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2016.
- Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Jindong Wang, Xixu Hu, Wenxin Hou, Hao Chen, Runkai Zheng, Yidong Wang, Linyi Yang, Wei Ye, Haojun Huang, Xiubo Geng, Binxing Jiao, Yue Zhang, and Xing Xie. On the robustness of chatgpt: An adversarial and out-of-distribution perspective. *IEEE Data Eng. Bull.*, 47(1):48–62, 2024a.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhramil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Ross Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024a. URL <https://arxiv.org/abs/2409.12122>.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024b.
- Edward Yeo, Yuxuan Tong, Xinyao Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in LLMs. In *ICLR 2025 Workshop on Navigating and Addressing Data Problems for Foundation Models*, 2025. URL <https://openreview.net/forum?id=AgtQlhMQ0V>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Lifan Yuan, Yangyi Chen, Ganqu Cui, Hongcheng Gao, FangYuan Zou, Xingyi Cheng, Heng Ji, Zhiyuan Liu, and Maosong Sun. Revisiting out-of-distribution robustness in NLP: Benchmarks, analysis, and LLMs evaluations. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.

- Zhecheng Yuan, Tianming Wei, Langzhe Gu, Pu Hua, Tianhai Liang, Yuanpei Chen, and Huazhe Xu. Hermes: Human-to-robot embodied learning from multi-source motion data for mobile dexterous manipulation. *arXiv preprint arXiv:2508.20085*, 2025.
- Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Dianyu Zhong, Yiqin Yang, Ziyong Zhang, Yuhua Jiang, Bo XU, and Qianchuan Zhao. Maximum next-state entropy for efficient reinforcement learning, 2024. URL <https://openreview.net/forum?id=0G6rRLYcxm>.
- Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.
- Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pp. 1433–1438. Chicago, IL, USA, 2008.



## A IMPLEMENTATION DETAILS

### A.1 IMPLEMENTATION DETAILS OF CONTINUOUS CONTROL TASKS

#### A.1.1 CODE IMPLEMENTATION OF ERA IN CONTINUOUS CONTROL

Listing 1: Original Implementation

```
# Original implementation from the
# jaxrl codebase, suggested by
# Ilya
# log_std_min, log_std_max: bounds
# for log standard deviation
# action_dim: dimension of the
# action space
# pre_stds: direct output from the
# actor network
log_stds = log_std_min + (
    log_std_max - log_std_min) *
    0.5 * (1 + nn.tanh(pre_stds))
```

Listing 2: ERA Implementation

```
# h_0: target entropy, can be a
# fixed value or a learnable
# parameter
# action_dim: dimension of the
# action space
k = - self.action_dim * (
    log_std_max + h_0 + jnp.log(jnp
    .sqrt(2 * jnp.pi * jnp.e)))
log_stds = k * nn.softmax(pre_stds,
    axis = -1) + log_std_max
log_stds = jax.clip(log_stds, self.
    log_std_min, self.log_std_max)
```

Figure 6: Comparison of the activation function at the actor’s output.

We provide the following JAX implementation snippet of ERA for the reader’s reference, where  $h_0$  is the target entropy ( $\mathcal{H}_0'$  in Eq. 11), which can be a constant (e.g.,  $-action\_dim/2$ ) or a learnable parameter. The terms  $log\_std\_min$  and  $log\_std\_max$  represent the lower and upper bounds of the log standard deviation, respectively;  $action\_dim$  is the dimension of the action space; and  $pre\_stds$  refers to the raw output of the actor network.

#### A.1.2 ENVIRONMENTS

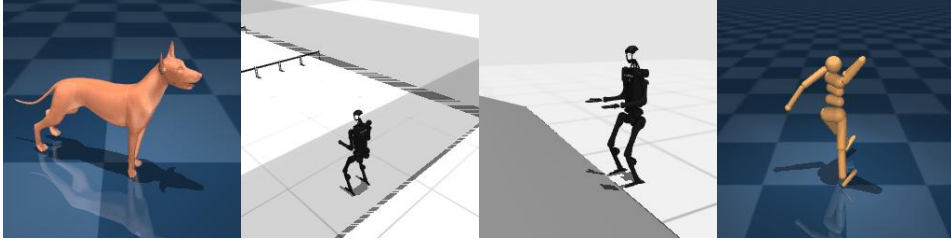


Figure 7: **Visualization of some continuous control environments used in our experiments.** From left to right: dog-run (DMC), h1-hurdle-v0 (HumanoidBench), h1hand-slide-v0 (HumanoidBench), humanoid-walk (DMC)

Our evaluation of ERA spans a diverse set of continuous control tasks from three established benchmarks: Mujoco Gym (Todorov et al., 2012), DeepMind Control Suite (DMC) (Tassa et al., 2018), and HumanoidBench (Sferrazza et al., 2024). For the Mujoco Gym and DMC environments, we utilized their standard, unmodified configurations. For HumanoidBench, we introduced specific modifications for certain agents.

For experiments involving SAC and OBAC on HumanoidBench, we implemented an action repeat of 2 and disabled episode termination. These adjustments were necessary because the standard tasks proved exceedingly challenging for a baseline SAC agent, as demonstrated in Figure 8. Conversely, for the FastSAC agent, which is capable of solving the original tasks, we used the standard HumanoidBench environments without these modifications.

For our comparison against TD-MPC2 on DMC environments, we used the performance data reported in the original manuscript. We therefore adhered to their experimental setup, which includes an action repeat of 2.

For main results and training curves, we report results over 10 random seeds for SAC, OBAC, and FastSAC, 5 seeds for PPO, and 3 seeds for TD-MPC2, matching the number provided in its original publication.

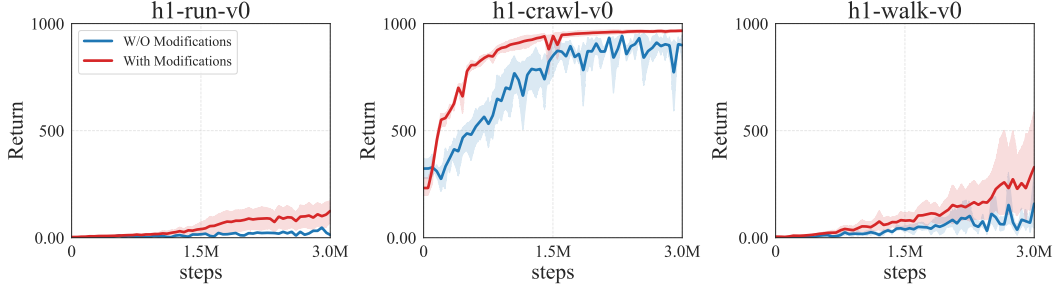


Figure 8: **Ablation of Environment Modifications for HumanoidBench.** Performance comparison of a standard SAC agent on three challenging HumanoidBench tasks with and without our modified settings (action repeat of 2 and disabled termination). The significant performance gap justifies using these modified settings for our main SAC-based experiments.

The action, observation spaces and maximal episode length of the respective environments are shown in Table 4 and Table 5.

Table 4: List of tasks from DeepMind Control and MetaWorld on which the agents were ablated. The table also contains the dimensions of action, observation space and maximal episode length.

| Task             | Observation dimension | Action dimension | Max episode length |
|------------------|-----------------------|------------------|--------------------|
| DEEPMIND CONTROL |                       |                  |                    |
| Dog-Trot         | 223                   | 38               | 1000               |
| Dog-Walk         | 223                   | 38               | 1000               |
| Dog-Run          | 223                   | 38               | 1000               |
| Humanoid-Run     | 67                    | 24               | 1000               |
| Humanoid-Walk    | 67                    | 24               | 1000               |
| Humanoid-Stand   | 67                    | 24               | 1000               |
| MUJOCO GYM       |                       |                  |                    |
| HalfCheetah-v4   | 17                    | 6                | 1000               |
| Ant-v4           | 27                    | 8                | 1000               |
| Hopper-v4        | 11                    | 3                | 1000               |
| Walker2d-v4      | 17                    | 6                | 1000               |

### A.1.3 PSEUDO CODE OF SAC-ERA

To better illustrate the role of our method within the algorithmic framework, we present the pseudocode for a representative example, the Soft Actor-Critic (SAC) algorithm, adapted with ERA in Algorithm 1.

### A.1.4 HYPERPARAMETERS

We present the hyperparameters used in our experiments with SAC and PPO in Table 6

Our implementations of SAC and OBAC are heavily inspired by the official `jaxrl` repository (Kostrikov, 2021). For the network design, we follow the insights from Nauman et al. (2024) and incorporate LayerNorm (Ba et al., 2016) into the neural networks.

Our OBAC implementation is built upon the codebase provided by Kang et al. (2025). It shares the same fundamental hyperparameters as our SAC implementation, with the behavior cloning weight set to  $1 \times 10^{-3}$ .

Table 5: List of tasks from HumanoidBench on which the agents were ablated. The table also contains the dimensions of action, observation space and maximal episode length.

| Task                     | Observation dimension | Action dimension | Max episode length |
|--------------------------|-----------------------|------------------|--------------------|
| h1-walk-v0               | 51                    | 19               | 500                |
| h1-run-v0                | 51                    | 19               | 500                |
| h1-stand-v0              | 51                    | 19               | 500                |
| h1-hurdle-v0             | 51                    | 19               | 500                |
| h1-stair-v0              | 51                    | 19               | 500                |
| h1-crawl-v0              | 51                    | 19               | 500                |
| h1hand-balance_simple-v0 | 164                   | 61               | 1000               |
| h1hand-hurdle-v0         | 151                   | 61               | 1000               |
| h1hand-pole-v0           | 151                   | 61               | 1000               |
| h1hand-push-v0           | 163                   | 61               | 1000               |
| h1hand-stair-v0          | 151                   | 61               | 1000               |
| h1hand-slide-v0          | 151                   | 61               | 1000               |
| h1hand-walk-v0           | 151                   | 61               | 1000               |
| h1hand-run-v0            | 151                   | 61               | 1000               |

Table 6: Comparison of hyperparameters for SAC and PPO.

| Hyperparameter                          | SAC                    | PPO                |
|-----------------------------------------|------------------------|--------------------|
| <i>Optimizer Settings</i>               |                        |                    |
| Actor optimizer                         |                        | Adam               |
| Actor learning rate                     |                        | $3 \times 10^{-4}$ |
| Critic optimizer                        | AdamW                  | Adam               |
| Critic learning rate                    |                        | $3 \times 10^{-4}$ |
| Temperature learning rate               | $3 \times 10^{-4}$     | —                  |
| Adam epsilon                            | —                      | $1 \times 10^{-5}$ |
| Gradient clipping                       | —                      | 0.5                |
| <i>Network Architecture</i>             |                        |                    |
| Actor/Critic network                    |                        | 3-layer MLP        |
| Hidden layer dimensions                 | (512, 512)             | (64, 64)           |
| Activation function                     | ReLU                   | Tanh               |
| LayerNorm                               | True                   | False              |
| <i>Algorithm Hyperparameters</i>        |                        |                    |
| Discount factor ( $\gamma$ )            |                        | 0.99               |
| Replay buffer size                      | $1 \times 10^6$        | —                  |
| Polyak averaging coefficient ( $\tau$ ) | 0.005                  | —                  |
| Initial temperature ( $\alpha$ )        | 1.0                    | —                  |
| Target entropy ( $\mathcal{H}_0$ )      | $-\dim(\mathcal{A})/2$ | —                  |
| Gradient steps per env. step            | 2                      | —                  |
| Random exploration steps                | 5,000                  | —                  |
| GAE parameter ( $\lambda$ )             | —                      | 0.95               |
| PPO clip ratio                          | —                      | 0.2                |
| Entropy coefficient                     | —                      | 0.01               |
| Batch size                              | 256                    | 2048               |
| Mini-batch size                         | —                      | 64                 |

For the PPO and PPO-ERA experiments, our implementation is based on the publicly available codebase of Li (2022). We use target entropy of  $-0.3\mathcal{A}$  for main experiments on PPO-ERA.

For the TD-MPC2 baseline, we utilize the official implementation provided by the original authors. The results for comparison are also directly sourced from those reported in the official repository. We use target entropy of  $-\mathcal{A}$  for main experiments on TD-MPC2-ERA.

Similarly, our implementations of FastTD3 and FastSAC are based on the official codebases provided by their respective authors. We note that our construction of FastSAC-ERA differs from the method described in the original paper; these differences are detailed in Section A.1.5.

#### A.1.5 FASTSAC-ERA

The FastTD3 (Seo et al., 2025) framework demonstrated the potential of applying off-policy RL methods to massively parallel RL scenarios, achieving excellent performance on HumanoidBench.

Authors of FastTD3 also provided a FastSAC implementation, which replaced the mixed noise mechanism in FastTD3 with the standard entropy maximization objective from Soft Actor-Critic (SAC). However, they noted that this approach yielded unstable results, and hypothesized that maximizing action entropy in high-dimensional action spaces might be inherently challenging.

To address this issue, we investigated a solution based on minimal modification to the original FastTD3. Our approach, named FastSAC-ERA, is derived from FastTD3 by retaining its noise mechanism while removing the Delayed Policy Updates and incorporating an entropy constraint via ERA implementation. This method achieved performance superior to that of FastTD3.

In practice, our implementation was built directly upon the official FastTD3 codebase. The only modifications were the removal of Delayed Policy Updates and the addition of the ERA implementation at the actor’s output. All other hyperparameters and implementation details were kept identical to the original FastTD3 configuration.

### A.2 IMPLEMENTATION DETAILS OF IMAGE CLASSIFICATION

#### A.2.1 CODE IMPLEMENTATION OF ERA IN IMAGE CLASSIFICATION

Listing 3: ERA Implementation in Image Classification

```
class ERA(nn.Module):
    def __init__(self, C_H: float, n_dims: int):
        super().__init__()
        self._tau = 4.
        self.C_H = C_H
        self.n_dims = n_dims

        self.upper_bound = math.log(self._tau) / self._tau
        assert C_H >= self.upper_bound
        self.slope = (self.upper_bound - C_H / n_dims) / (1 - 1 / n_dims)
        self.b = (C_H - self.slope) / n_dims

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        x: logits before softmax, shape (...)
        return: adjusted logits before softmax, shape (...)
        """
        h = self.slope * x.softmax(dim=-1) + self.b
        u = -1 - torch.log(h)
        new_logits = (-1 - torch.sqrt(2 * u) - 3/4 * u).to(x.dtype)

        max_values = torch.max(x, dim=-1, keepdim=True).values.detach()
        x = x - max_values
        min_values = torch.min(new_logits, dim=-1, keepdim=True).values.detach()
        new_logits = new_logits - min_values

        return new_logits
```

We provide the implementation of ERA for image classification tasks in Listing 3. In the code,  $C_H$  corresponds to  $C_{\mathcal{H}_0}$  defined in Eq. 15, and  $n\_dims$  denotes the number of classes. We set  $\tau = 4$  in our implementation without performing any tuning for this parameter.

---

**Algorithm 1** Soft Actor-Critic (SAC) with ERA

---

- 1: **Initialize:** actor parameters  $\theta$ , critic parameters  $\phi_1, \phi_2$ .
- 2: **Initialize:** target network parameters  $\phi'_1 \leftarrow \phi_1, \phi'_2 \leftarrow \phi_2$ .
- 3: **Initialize:** replay buffer  $\mathcal{D}$ .
- 4: **Hyperparameters:** learning rates  $\lambda_\pi, \lambda_Q$ , target entropy  $\mathcal{H}_0$ , Polyak coefficient  $\tau$ .
- 5: **for** each training step **do**
- 6:   Sample action from the policy:  $a_t \sim \pi_\theta(\cdot|s_t)$ .
- 7:   Execute action  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ .
- 8:   Store transition  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer  $\mathcal{D}$ .
- 9:   Sample a random minibatch of transitions  $B = \{(s, a, r, s')\}$  from  $\mathcal{D}$ .
- 10:   *// Update the Q-functions (critics)*
- 11:   Sample next actions:  $a' \sim \pi_\theta(\cdot|s')$ .
- 12:   Compute the target Q-value by taking the minimum of the two target critics:

$$Q'_{\text{target}}(s', a') \leftarrow \min_{i=1,2} Q_{\phi'_i}(s', a')$$

- 13:   Compute the soft Q-target  $y$  (matches Eq. 13):

$$y \leftarrow r + \gamma Q'_{\text{target}}(s', a')$$

- 14:   Update both critics by one step of gradient descent using the loss from Eq. 7:

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,y) \in B} \frac{1}{2} (Q_{\phi_i}(s, a) - y)^2 \quad \text{for } i = 1, 2$$

- 15:   *// Update the policy (actor)*
- 16:   Sample new actions for the policy update (using reparameterization trick):  $\tilde{a} \sim \pi_\theta(\cdot|s)$ .
- 17:   Compute Q-values for the new actions using the minimum of the two critics:

$$Q_{\min}(s, \tilde{a}) \leftarrow \min_{i=1,2} Q_{\phi_i}(s, \tilde{a})$$

- 18:   Update the policy by one step of gradient ascent to maximize the objective from Eq. 14:

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_{\min}(s, \tilde{a})$$

- 19:   *// Update target networks using Polyak averaging*
  - 20:    $\phi'_i \leftarrow \tau \phi_i + (1 - \tau) \phi'_i$  for  $i = 1, 2$
  - 21: **end for**
- 

### A.2.2 TRAINING SETUP

Our training for ImageNet was completed on 4 A100 GPUs, and we report the 95% confidence interval calculated from the dataset. For CIFAR-10, which requires less computation, we trained three separate runs on 3 machines, each with 4 A40 GPUs, and report the confidence interval computed from these three results to ensure maximum reproducibility.

### A.2.3 COMMANDS USED FOR EXPERIMENTS

We provide two main commands used for training in image classification. The two commands delineate the training procedures for our models under two distinct settings: one incorporating data augmentation and the other without it. The training commands were sourced directly from the reference ImageNet training script within the timm library. We employed this identical set of commands for training on both the ImageNet and CIFAR-10 datasets without any dataset-specific hyperparameter tuning to ensure a consistent experimental setup.

Listing 4: Command to launch training with data augmentation.

---

```
./distributed_train.sh 4 --data-dir ../data --dataset torch/cifar10 --
  ↳ dataset-download -b 64 --model resnet50 --sched cosine --epochs 200
  ↳ --lr 0.05 --amp --remode pixel --reprob 0.6 --aug-splits 3 --aa
  ↳ rand-m9-mstd0.5-inc1 --resplit --split-bn --jsd --dist-bn reduce
```

Listing 5: Command to launch training without data augmentation (baseline).

```
./distributed_train.sh 4 --data-dir ../data --dataset torch/cifar10 --
  ↳ dataset-download -b 64 --model resnet50 --sched cosine --epochs 200
  ↳ --lr 0.05 --amp --dist-bn reduce
```

### A.3 IMPLEMENTATION DETAILS OF LLM TRAINING

#### A.3.1 CODE IMPLEMENTATION OF ERA IN LLM

We provide the core implementation of ERA in LLM in Listing 6. In the code, `era_lb`, `era_ub` and `era_k` corresponds to  $\omega_{\text{low}}$ ,  $\omega_{\text{high}}$ ,  $k$  defined in Eq. 16, respectively. In the first training stage, we further apply a top- $k$  filter (retaining the 20 largest logits) within the `logprobs_from_logits` function to enhance training stability. Additionally, in practice, we found that the advantage scaling does not affect model performance, so we did not implement it in our code.

Listing 6: ERA Implementation in LLM

```
length = response_mask.sum(dim=-1)
k_per_sample = (0.2 * length).long().clamp(min=1)

mean_top_entropy = []
masked_entropy = entropy.masked_fill(~response_mask.bool(), float("-inf"))
for b in range(entropy.size(0)):
    k = k_per_sample[b].item()
    top_entropy_b, _ = torch.topk(masked_entropy[b], k)
    mean_top_entropy.append(top_entropy_b.mean())

mean_top_entropy = torch.stack(mean_top_entropy).unsqueeze(-1)
cond_A = (mean_top_entropy < era_lb) & (advantages > 0)
cond_B = (mean_top_entropy > era_ub) & (advantages > 0)

logits[cond_A] = logits[cond_A] * era_k
logtis[cond_B] = logits[cond_B] / era_k

log_prob = logprobs_from_logits(logits)
```

#### A.3.2 HYPERPARAMETERS

For GRPO, GRPO w/ 20% Forking Tokens, ERA, we use a training batch size of 256 and a mini batch size of 256 in the verl configuration, which results in a on-policy setting. For KL-Cov and Clip-Cov, we use a training batch size of 256 and a mini batch size of 32. The learning rate is  $10^{-6}$  and no learning rate warm-up or scheduling is applied. We also utilize dynamic sampling to enhance training efficiency. Since our setting is on-policy, the clip ratio is irrelevant. The maximum response length is 8192 with no overlong reward shaping. For Qwen2.5-Math-1.5B, we use MATH problems of levels 3–5 as the training set in this experiment since DAPO-Math-17K is too difficult.

The hyperparameters of ERA are fixed to  $\omega_{\text{low}} = 0.45$ ,  $\omega_{\text{high}} = 3.0$ , and  $k = 2$  across all settings, without any tuning. These values are chosen with reference to the initial entropy of the model,  $H_{\text{resp}} \approx 1.5$ , such that  $\omega_{\text{low}}$  and  $\omega_{\text{high}}$  lie below and above this value, respectively. The only exception is in the second training stage of ERA for the Qwen2.5-Math-7B model, where we set  $\omega_{\text{low}} = 0.2$ ,  $\omega_{\text{high}} = +\infty$ , and  $k = 2$ .



## B PROOFS AND DERIVATIONS

### B.1 PROOF OF ENTROPY BOUND IN CONTINUOUS SPACE

Recall the continuous form of ERA:

$$\mu' = \mu, \quad \sigma' = \exp \left[ \max \left( \log \sigma_{\max} + (\mathcal{H}'_0 - D \log \sqrt{2\pi e} - D \log \sigma_{\max}) \frac{e^{\hat{\sigma}_i}}{\sum_{j=1}^D e^{\hat{\sigma}_j}}, \log \sigma_{\min} \right) \right]$$

Here,  $\mathcal{H}'_0$  is the target entropy for the final policy  $\mathcal{H}_0$  plus a residual entropy term  $\delta \geq 0$  to account for the bounding bias. The residual entropy term  $\delta$  is defined as the minimum additional entropy required to ensure that the final entropy, after transformation of the Gaussian distribution, is at least  $\mathcal{H}_0$ . And forms for  $\delta$  in both squashed Gaussian and truncated Gaussian cases are given by Eq. 19 and Eq. 20, respectively.

$$\delta_{\text{tanh}} = -\mathbb{E}_{s \sim \rho_\pi, u \sim \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))} \left[ \sum_{i=1}^D \log(1 - \tanh(u_i)^2) \right] \quad (19)$$

$$\delta_{\text{TN}} = -\mathbb{E}_{s \sim \rho_\pi} \left[ \sum_{i=1}^D \left( \log Z_i(s) - \frac{\beta_i(s)\phi(\beta_i(s)) - \alpha_i(s)\phi(\alpha_i(s))}{2Z_i(s)} \right) \right] \quad (20)$$

In practice, we can learn a  $\hat{\delta} \geq 0$  as a state-independent parameter by minimizing loss in Eq. 12, which provides a straightforward learning mechanism that is agnostic to the specific form of the distribution.

$$\mathcal{L}(\hat{\delta}) = \mathbb{E}_{s \sim \mathcal{D}} \left[ \hat{\delta} (\mathcal{H}[\pi(\cdot|s)] - \mathcal{H}_0) \right]$$

We directly adopt the loss function from the automated entropy adjustment in SAC (Haarnoja et al., 2018). We note that the optimization objective in our method is identical to that in SAC, and thus it can be formulated as a nearly identical dual problem, formed in Eq. 21. The convergence of  $\hat{\delta}$  is therefore guaranteed by strong duality.

$$\max_{\pi} \mathbb{E}_{(s,a) \sim \rho_\pi} [r(s,a)] = \min_{\hat{\delta} \geq 0} \max_{\pi} \left( \mathbb{E}_{(s,a) \sim \rho_\pi} [r(s,a)] - \hat{\delta} \log \pi(a|s) - \hat{\delta} \mathcal{H}_0 \right) \quad (21)$$

Now we can prove that the entropy of the final policy is guaranteed to be at least  $\mathcal{H}_0$ .

**Proposition 1.** *Given a target entropy  $\mathcal{H}_0$  and a residual entropy  $\hat{\delta} \geq \delta$ , the policy defined by Eq. 11 has entropy  $\mathcal{H}(\pi) \geq \mathcal{H}_0$ , and  $\sigma'$  is bounded within  $[\sigma_{\min}, \sigma_{\max}]$ .*

*Proof.* We only need to show that the entropy of the Gaussian before transformation is at least  $\mathcal{H}_0 + \delta$  to prove the entropy bound. The entropy of a Gaussian distribution  $\mathcal{N}(\mu, \Sigma)$  is given by  $\mathcal{H}(\mathcal{N}(\mu, \Sigma)) = \frac{D}{2} \log(2\pi e) + \frac{1}{2} \log |\Sigma|$ . Since  $\Sigma$  is diagonal, we have  $\log |\Sigma| = \sum_{i=1}^D \log \sigma_i^2 = 2 \sum_{i=1}^D \log \sigma_i$ . Therefore, the entropy can be expressed as:

$$\mathcal{H}(\mathcal{N}(\mu, \Sigma)) = \frac{D}{2} \log(2\pi e) + \sum_{i=1}^D \log \sigma_i \quad (22)$$

Hence,  $\mathcal{H}(\mathcal{N}(\mu, \Sigma)) \geq \mathcal{H}_0 + \delta$  is equivalent to:

$$\sum_{i=1}^D \log \sigma_i \geq \mathcal{H}_0 + \delta - \frac{D}{2} \log(2\pi e) \quad (23)$$

For ERA-transformed Gaussian distribution, we have:

$$\begin{aligned} \sum_{i=1}^D \log \sigma_i &\geq \sum_{i=1}^D \left[ \log \sigma_{\max} + (\mathcal{H}'_0 - D \log \sqrt{2\pi e} - D \log \sigma_{\max}) \frac{e^{\hat{\sigma}_i}}{\sum_{j=1}^D e^{\hat{\sigma}_j}} \right] \\ &= D \log \sigma_{\max} + (\mathcal{H}'_0 - D \log \sqrt{2\pi e} - D \log \sigma_{\max}) \\ &= \mathcal{H}'_0 - \frac{D}{2} \log(2\pi e) \end{aligned} \quad (24)$$

Since  $\mathcal{H}'_0 = \mathcal{H}_0 + \hat{\delta} \geq \mathcal{H}_0 + \delta$ , the condition in Eq. 23 is satisfied. Additionally, the use of the max function in Eq. 11 ensures that  $\sigma'$  is bounded within  $[\sigma_{\min}, \sigma_{\max}]$ . This completes the proof.  $\square$

## B.2 PROOF OF ENTROPY BOUND IN DISCRETE SPACE

Recall the discrete form of ERA:

$$z' = h^{-1} \left[ \max \left( \frac{\log \tau}{\tau} + \left( C_{\mathcal{H}_0} - n \frac{\log \tau}{\tau} \right) \frac{1}{D-1} \left( 1 - \frac{e^{z_i}}{\sum_{j=1}^D e^{z_j}} \right), 0 \right) \right]$$

Before we delve into the proof of its entropy bound, we first provide some insights into the design of ERA in the context of vision tasks. To adapt the entropy constraint function from continuous spaces for discrete domains, our initial idea was to have the network output the entropy of individual components rather than their logits. However, this direct approach is problematic because the function  $H(p) = -p \ln p$  is non-monotonic over the interval  $[0, 1]$ . This ambiguity means a given entropy value cannot be uniquely mapped back to its corresponding probability; for instance, an entropy of 0 could correspond to a probability of either 0 or 1.

To resolve this ambiguity, we introduce a scaling factor  $\tau > e$  and consider a " $\tau$ -divided distribution," where each probability is scaled down by  $\tau$ . By selecting  $\tau > e$ , we ensure that the function  $-p \ln p$  is strictly monotonically increasing on the interval  $[0, 1/\tau]$ . This establishes a one-to-one mapping, allowing for the unique recovery of a probability value from its entropy within this restricted range. Therefore, our network is designed to output the entropy of this  $\tau$ -divided distribution. We then map these entropy values back to the probability space using the inverse function,  $h^{-1}$ . As  $h^{-1}$  lacks a closed-form solution, we utilize a numerical approximation. A final normalization step is required because the resulting probabilities from this inverse mapping do not inherently sum to one.

Crucially, we have proven that the entropy loss during this normalization process is bounded. By leveraging the continuous-space entropy constraint function to ensure the initial output entropy is above a threshold  $C_{\mathcal{H}_0}$ , we can guarantee that the entropy of the final discrete distribution will also exceed  $C_{\mathcal{H}_0}$ . This constitutes the core mechanism behind the implementation of ERA in discrete spaces.

**Proposition 2.** *Given a target entropy  $\mathcal{H}_0$  and a hyperparameter  $\tau \geq e$ , the policy defined by Eq. 15 has entropy  $\mathcal{H}(\pi) \geq \mathcal{H}_0$ .*

*Proof.* We denote  $\kappa = \max(\frac{\log \tau}{\tau} + (C_{\mathcal{H}_0} - n \frac{\log \tau}{\tau}) \frac{1}{D-1} (1 - \frac{e^{z_i}}{\sum_{j=1}^D e^{z_j}}), 0)$ . Similar to the continuous case, we have  $\kappa$  bounded within  $[0, \frac{\log \tau}{\tau}]$  and  $\sum_{i=1}^D \kappa_i \geq C_{\mathcal{H}_0}$ . We denote the probability of the final softmax policy as  $p = \text{softmax}(z') = \frac{e^{z'_i}}{\sum_{j=1}^D e^{z'_j}}$ . Then we have:

$$\begin{aligned} \mathcal{H}(\pi) &= - \sum_{i=1}^D p_i \log p_i \\ &= - \frac{\sum_{i=1}^D e^{h^{-1}(\kappa_i)} h^{-1}(\kappa_i)}{\sum_{j=1}^D e^{h^{-1}(\kappa_j)}} + \log \left( \sum_{j=1}^D e^{h^{-1}(\kappa_j)} \right) \\ &\geq 1 + \log \left( - \sum_{i=1}^D e^{h^{-1}(\kappa_i)} h^{-1}(\kappa_i) \right) \end{aligned} \tag{25}$$

Recall that  $h = \ln \circ (-x \ln x)$ , so  $h^{-1} = (-x \ln x)^{-1} \circ \ln^{-1}$ . Hence we have:

$$\begin{aligned} \mathcal{H}(\pi) &\geq 1 + \log \left( - \sum_{i=1}^D e^{h^{-1}(\kappa_i)} h^{-1}(\kappa_i) \right) \\ &= 1 + \log \left( \sum_{i=1}^D \kappa_i \right) \geq 1 + \log(C_{\mathcal{H}_0}) = \mathcal{H}_0 \end{aligned} \tag{26}$$

□

### B.3 PROOF OF ENTROPY BOUND IN LLMs

Recall the definition of the ERA instantiation for LLMs:

$$z' = \begin{cases} kz & H_{\text{resp}} < \omega_{\text{low}}, A_t > 0, \\ z & \omega_{\text{low}} \leq H_{\text{resp}} \leq \omega_{\text{high}}, A_t < 0 \text{ or } A_t > 0, \\ \frac{1}{k}z & H_{\text{resp}} > \omega_{\text{high}}, A_t > 0, \end{cases}$$

and

$$A'_t = \begin{cases} \frac{1}{k}A_t & H_{\text{resp}} < \omega_{\text{low}}, A_t > 0, \\ A_t & \omega_{\text{low}} \leq H_{\text{resp}} \leq \omega_{\text{high}}, A_t < 0 \text{ or } A_t > 0, \\ kA_t & H_{\text{resp}} > \omega_{\text{high}}, A_t > 0, \end{cases}$$

where  $z$  are the logits,  $A_t$  the advantages, and  $H_{\text{resp}}$  is the average entropy of the top 20% of tokens with the highest entropy in the response.

These transformations are applied after sampling. The modified policy-gradient objective is therefore

$$J(\theta) = \mathbb{E}_t[\mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_t)} \log \pi'_\theta(a_t|s_t) A'_t]$$

Intuitively, when the entropy is too low, ERA sharpens the policy; when it is too high, ERA flattens it. By rescaling the advantages of modified tokens, we show below that ERA is equivalent to augmenting the vanilla policy-gradient objective with an adaptive KL regularizer. This KL term guarantees that the entropy of responses remains in the interval  $[\omega_{\text{low}}, \omega_{\text{high}}]$ , preventing entropy collapse. Under mild assumptions, we derive a positive entropy lower bound.

Fixing the state  $s_t$ , denote  $\pi_a = \pi_\theta(a|s_t)$ ,  $\pi'_a = \pi'_\theta(a|s_t)$ , and  $A_a$  the advantage of action  $a$ . The entropy is  $H = -\sum_a \pi_a \log \pi_a$ . We first derive the gradient of the entropy.

**Lemma 1.**

$$\begin{aligned} \frac{\partial H}{\partial z_a} &= \sum_{a'} -\frac{\partial \log \pi_{a'}}{\partial z_a} (\pi_{a'} \log \pi_{a'} + \pi_{a'}) \\ &= \sum_{a'} -([a = a'] - \pi_a) (\pi_{a'} \log \pi_{a'} + \pi_{a'}) \\ &= -\pi_a (\log \pi_a + H). \end{aligned} \tag{27}$$

We also define the  $\pi$ -weighted covariance that will be used later:

**Definition 1.** Define the  $\pi$ -weighted covariance for two vectors  $x = (x_a)$ ,  $y = (y_a)$  by

$$\text{Cov}_\pi(x, y) = \sum_a \pi_a x_a y_a - \left( \sum_a \pi_a x_a \right) \left( \sum_a \pi_a y_a \right).$$

Now we show our main result:

**Proposition 3.** Let  $\pi_\theta$  be the base policy and  $\pi'_\theta$  the ERA-adjusted policy from Eq. equation 16. Suppose that:

- (i) (Logit approximation) The change in entropy can be approximated by treating logits  $z$  as the effective policy parameters and using first-order (infinitesimal) sensitivity of entropy w.r.t.  $z$ .
- (ii) (Positive advantage mass) The aggregated positive advantage restricted to the tokens considered in  $H_{\text{resp}}$ ,

$$C(s_t) = \sum_{a, A_a > 0} \pi_a A_a,$$

satisfies  $C(s_t) \geq \gamma$  for some  $\gamma > 0$ .

- (iii) (Bounded response entropy) In some intermediate point of the training process,  $H_{\text{resp}}$  has a lower bound  $H_{\min}$  and an upper bound  $\omega_{\text{high}}$ .
- (iv) (Bounded PG-induced entropy decrease) We assume the vanilla policy-gradient term's expected effect on entropy is bounded as

$$\mathbb{E}[\text{Cov}_{\pi}(\pi_a A_a, \log \pi_a)] \leq \alpha H,$$

for some  $\alpha \geq 0$ , where  $H$  denotes the entropy of the current policy  $\pi$ .

- (v) (Bounded KL-induced entropy decrease) We assume there exists a constant  $B_k > 0$  (that depends on  $k$  and  $H_{\min}$ ) such that

$$\text{Cov}_{\pi}(\pi'_a - \pi_a, \log \pi_a) \geq B_k H,$$

If  $\gamma B_k - \alpha > \beta$  for  $\beta > 0$ , then there exists a constant  $\mathcal{H}_0 > 0$  such that the response entropy satisfies

$$\mathbb{E}[H_{\text{resp}}] \geq \mathcal{H}_0$$

under ERA updates.

*Proof.* When  $H_{\text{resp}} < \omega_{\text{low}}$ , ERA sharpens positively advantaged actions. Following the derivation, the ERA-adjusted gradient satisfies

$$\begin{aligned} & \frac{\partial}{\partial z_a} \mathbb{E}_{a' \sim \pi} \log \pi'_{a'} A'_{a'} \\ &= \frac{\partial}{\partial z_a} \mathbb{E}_{a' \sim \pi} \left( [A_{a'} > 0] \log \pi'_{a'} \frac{1}{k} A_{a'} + [A_{a'} < 0] \log \pi_{a'} A_{a'} \right) \\ &= \mathbb{E}_{a' \sim \pi} \left( [A_{a'} > 0] \frac{\partial \log \pi'_{a'}}{\partial z'_a} \frac{\partial z'_a}{\partial z_a} \frac{1}{k} A_{a'} + [A_{a'} < 0] \frac{\partial \log \pi_{a'}}{\partial z_a} A_{a'} \right) \\ &= \mathbb{E}_{a' \sim \pi} ([A_{a'} > 0]([a' = a] - \pi'_{a'}) A_{a'} + [A_{a'} < 0]([a' = a] - \pi_{a'}) A_{a'}) \\ &= \pi_a A_a - \pi'_a \sum_{a', A_{a'} > 0} \pi_{a'} A_{a'} - \pi_a \sum_{a', A_{a'} < 0} \pi_{a'} A_{a'}, \end{aligned} \quad (28)$$

Since the expectation of advantage is zero, and we have defined  $C(s_t) = \sum_{a', A_{a'} > 0} \pi_{a'} A_{a'}$ , yielding

$$\frac{\partial}{\partial z_a} \mathbb{E}_{a' \sim \pi} \log \pi'_{a'} A'_{a'} = \pi_a A_a - C(s_t)(\pi'_a - \pi_a). \quad (29)$$

For the vanilla policy-gradient loss, this reduces to

$$\frac{\partial}{\partial z_a} \mathbb{E}_{a' \sim \pi} \log \pi_{a'} A_{a'} = \pi_a A_a \quad (30)$$

Meanwhile, by a similar derivation, the gradient of the KL divergence is

$$\frac{\partial}{\partial z_a} \text{KL}[\pi' \parallel \pi] = -\frac{\partial}{\partial z_a} \mathbb{E}_{a' \sim \pi'} \log \pi_{a'} = \pi_a - \pi'_a. \quad (31)$$

Thus, by combining equation 29, equation 30 and equation 31, the ERA-adjusted objective can be written as

$$J'(\theta) = \mathbb{E}_t [\mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t)} \underbrace{\log \pi_{\theta}(a_t | s_t) A_t}_{J_{\text{PG}}} + \text{sg}(C(s_t)) \underbrace{\text{KL}[\pi'_{\theta}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]}_{J_{\text{KL}}}], \quad (32)$$

where the  $\text{sg}(\cdot)$  denotes the stop gradient operator. For the other case  $\omega_{\text{low}} \leq$  (we have assumed that  $H_{\text{resp}} \leq \omega_{\text{high}}$ , the same structure holds; only the definition of  $\pi'_{\theta}$  changes. Hence, ERA is equivalent

to a policy gradient objective augmented with an adaptive KL regularizer that sharpens or flattens the distribution depending on  $H_{\text{resp}}$  and also the value of  $C(s_t)$ .

We will evaluate the instantaneous directional derivative of entropy along these gradient directions (this corresponds to the first-order change in entropy under an infinitesimal step in the indicated direction).

Using equation 27, the first-order change of entropy caused by  $J_{\text{PG}}$  is

$$\begin{aligned}\Delta H_{\text{PG}} &= \sum_a \frac{\partial H}{\partial z_a} \cdot \pi_a A_a \\ &= \sum_a -\pi_a (\log \pi_a + H) \cdot \pi_a A_a \\ &= -\sum_a \pi_a^2 A_a (\log \pi_a + H) \\ &= -\text{Cov}_\pi(\pi_a A_a, \log \pi_a).\end{aligned}\tag{33}$$

By assumption (iv) this term is bounded below by  $-\alpha H$ :

$$\mathbb{E}[\Delta H_{\text{PG}}] \geq -\alpha H.$$

Thus the vanilla policy-gradient component can decrease entropy, but by no more than  $\alpha H$  in magnitude.

Similarly, the KL-term directional derivative is

$$\begin{aligned}\Delta H_{\text{KL}} &= \sum_a \frac{\partial H}{\partial z_a} \cdot (\pi_a - \pi'_a) \\ &= \sum_a -\pi_a (\log \pi_a + H) \cdot (\pi_a - \pi'_a) \\ &= \sum_a \pi_a (\pi'_a - \pi_a) (\log \pi_a + H) \\ &= \text{Cov}_\pi(\pi'_a - \pi_a, \log \pi_a)\end{aligned}\tag{34}$$

By assumption (v) we have  $\text{Cov}_\pi(\pi'_a - \pi_a, \log \pi_a) \geq B_k H$ . Using assumption (ii)  $C(s_t) \geq \gamma$  therefore yields

$$C(s_t) \Delta H_{\text{KL}} \geq \gamma B_k H.$$

Combining the two contributions,

$$\mathbb{E}[\Delta H] = \mathbb{E}[\Delta H_{\text{PG}} + C(s_t) \Delta H_{\text{KL}}] \geq -\alpha H + \gamma B_k H = (\gamma B_k - \alpha) H.$$

By the hypothesis  $\gamma B_k - \alpha > \beta$  we have  $\Delta H > \beta H$  whenever  $H > 0$  and  $H$  is in the sharpening regime. Thus, if  $H_{\text{resp}}$  drops below  $\omega_{\text{low}}$ , the ERA-induced update produces a positive first-order increase in entropy proportional to  $H_{\text{resp}}$ . Consequently the dynamics push  $H_{\text{resp}}$  upward until it leaves the sharpening regime (i.e., until  $H_{\text{resp}} \geq \omega_{\text{low}}$  or the KL-term no longer sharpens).

Formally, the expected change of total entropy is at least

$$\beta \mathbb{E}_{H_{\text{resp}} < \omega_{\text{low}}} [H_{\text{resp}}] - \alpha \mathbb{E}_{H_{\text{resp}} \geq \omega_{\text{low}}} [H_{\text{resp}}]\tag{35}$$

Applying Markov's inequality gives  $\Pr(H_{\text{resp}} \geq \omega_{\text{low}}) \leq \mu / \omega_{\text{low}}$ , where  $\mu = \mathbb{E}[H_{\text{resp}}]$ . Further, by assumption (iii):  $H_{\text{min}} \leq H_{\text{resp}} \leq \omega_{\text{high}}$ , we obtain the sufficient condition to make the expected entropy change positive:

$$\beta > \alpha \cdot \frac{\mu \omega_{\text{high}}}{(\omega_{\text{low}} - \mu) H_{\text{min}}}.$$

Then there exists a constant  $\mu$ , such that the expected change of total entropy is positive. Therefore by taking  $\mathcal{H}_0 = \mu$ ,  $H_{\text{resp}}$  is prevented from collapsing to zero and satisfies

$$H_{\text{resp}} \geq \mathcal{H}_0.$$

□

We now justify the assumptions made in Proposition 3.

- (i) The first assumption, namely approximating entropy differences by treating logits as policy parameters, is standard and also adopted by (Cui et al., 2025b).
- (ii) Recall that  $C(s_t) = \sum_{a, A_a > 0} \pi_a A_a$  measures the aggregated positive advantage, which reflects the “importance” of a token. Intuitively,  $C(s_t)$  indicates whether a token should remain explorative and thus be subject to entropy regularization. We assume that for important tokens,  $C(s_t)$  is uniformly bounded below by some constant  $\gamma > 0$ .
- (iii) Empirically, our training curves show that responses with  $H_{\text{resp}} > \omega_{\text{high}}$  vanish rapidly, and such cases contribute negligibly to the average entropy. This supports the assumption  $H_{\text{resp}} \leq \omega_{\text{high}}$ . Moreover, in the early stage of training, the highest entropy tokens (top 20%) contain a lot of exploratory tokens, exhibiting a large average entropy, motivating the assumption of a positive lower bound  $H_{\text{resp}} \geq H_{\text{min}}$ .
- (iv) It is provable that

$$\text{Cov}_{\pi}(\pi_a A_a, \log \pi_a) \leq H,$$

where  $H$  denotes the entropy. In practice this upper bound is rarely tight, and we assume instead a looser bound with a small constant  $\alpha \in (0, 1)$ .

- (v) In our regime, the entropy is low enough that the token with the largest probability dominates (with probability  $\geq 0.6$ ). In this setting, the covariance is large enough and is proportional to the entropy  $H$ .

In practice, the observed entropy lower bound is higher than the theoretical bound derived in Proposition 3, owing both to the looseness of the Markov inequality used in the derivation and to the fact that the tokens outside  $H_{\text{resp}}$  (bottom 80%) also get an entropy boost.

## C ADDITIONAL RESULTS

### C.1 ADDITIONAL RESULTS ON CONTINUOUS CONTROL TASKS

In this subsection, we provide additional experimental results on continuous control tasks to further validate the effectiveness of our proposed method, ERA, and to find more insights regarding entropy regularization in reinforcement learning.

#### C.1.1 TRUNCATED GAUSSIAN IS MORE STABLE THAN TANH GAUSSIAN

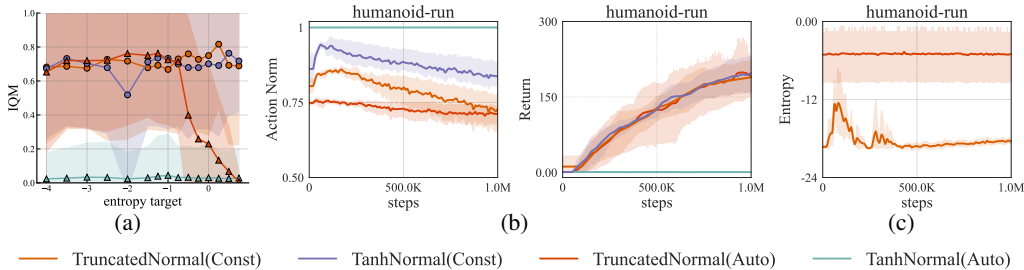


Figure 9: **Analysis of Policy Distributions.** Comparison of Truncated and Tanh Gaussian policies with varying  $\delta$  on DMC tasks. Target entropy represents the desired average entropy per action dimension. (a) The Truncated Gaussian exhibits greater stability across four DMC tasks. (b) For the Tanh Gaussian with a learned  $\delta$ , instability arises as action norms approach the boundary, causing training to collapse. (c) The Truncated Normal distribution’s entropy remains stable and well-controlled in both modes, shown here for a target entropy of -0.75.

We study the choice of policy distribution and the handling of its standard deviation,  $\delta$ . We compare a Truncated Gaussian against a Tanh-squashed Gaussian, each with a constant  $\delta$  (set to 0 in our experiments) and a learned  $\delta$ , using SAC on four hardest tasks from the DMC Dog & Humanoid suites (*dog-run*, *dog-trot*, *humanoid-run*, *humanoid-walk*) with 5 seeds and 1M environmental steps.



As shown in Figure 9, the Truncated Gaussian is significantly more stable. The Tanh Gaussian experiences catastrophic training failures when  $\delta$  is learned. Our analysis reveals that with the Tanh Gaussian, the action norm often approaches the distribution’s boundaries. This causes the learned  $\delta$  to grow explosively, creating a vicious cycle of instability as the policy attempts to output actions near the boundary while satisfying the entropy objective. This issue is absent in the Truncated Gaussian, which yields stable  $\delta$  values. Given that the performance difference between a learned and a constant  $\delta$  is minimal under the Truncated Gaussian, we adopt the constant  $\delta$  of 0 setting for its simplicity in main results.

### C.1.2 BATCH-LEVEL ENTROPY REGULARIZATION V.S. STATE-LEVEL ENTROPY REGULARIZATION

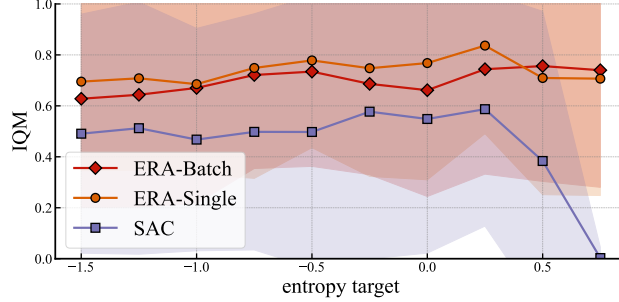


Figure 10: **Comparison between state-level and batch-level entropy regularization methods on DMC Dog & Humanoid suites.** Both methods outperform the SAC baseline.

In addition to the state-level entropy regularization method presented in the main paper, we also investigate a batch-level entropy regularization method, which directly constrains the expected entropy of the action distribution over  $\rho_\pi$ . Specifically, we modify the activation form of ERA in Eq. 11 to the form in Eq. 36.

$$\mu' = \mu, \quad \sigma' = \exp \left[ \max \left( \log \sigma_{\max} + \left( \frac{\mathcal{H}'_0}{D} - \log \sqrt{2\pi e} - \log \sigma_{\max} \right) \frac{e^{\hat{\sigma}_i}}{\bar{e}^{\hat{\sigma}}}, \log \sigma_{\min} \right) \right] \quad (36)$$

Where  $\bar{e}^{\hat{\sigma}} = \frac{1}{N} \sum_{i=1}^N e^{\hat{\sigma}_i}$  is the average of  $e^{\hat{\sigma}_i}$  over the batch. During training, we can calculate  $\bar{e}^{\hat{\sigma}}$  over the sampled batch, and during evaluation, we can use a running average of  $\bar{e}^{\hat{\sigma}}$  over the training process, which is similar to the running statistics in BatchNorm (Ioffe & Szegedy, 2015). We conduct an ablation study to compare the performance of state-level and batch-level entropy regularization methods on DMC Dog & Humanoid suites (*dog-run*, *dog-trot*, *humanoid-run*, *humanoid-walk*). As shown in Figure 10, both methods achieve similar performance, outperforming the SAC baseline. This indicates that in locomotion-dominated control tasks, which require high exploration due to the need for randomness but do not demand high precision, the difference between state-level and batch-level entropy regularization is minimal.

### C.1.3 SAC-ERA ON MUJOCO GYM ENVIRONMENTS

We also evaluate the performance of SAC-ERA on the classic Mujoco Gym environments, including *HalfCheetah-v4*, *Hopper-v4*, *Walker2d-v4*, *Ant-v4*, *Humanoid-v4*, *Swimmer-v4*, and compare it with the SAC baseline. Figure 11 shows the learning curves of SAC-ERA and SAC on these environments. Despite their massive performance gap on HumanoidBench, SAC-ERA demonstrates only slight advantages over SAC on Mujoco Gym environments. This may be due to the relatively low action space dimensionality in Mujoco environments, which reduces the impact of different constraint schemes. This finding suggests that modern algorithm design should shift focus from considering Mujoco to higher-dimensional action spaces, which can better evaluate algorithm performance in complex environments.

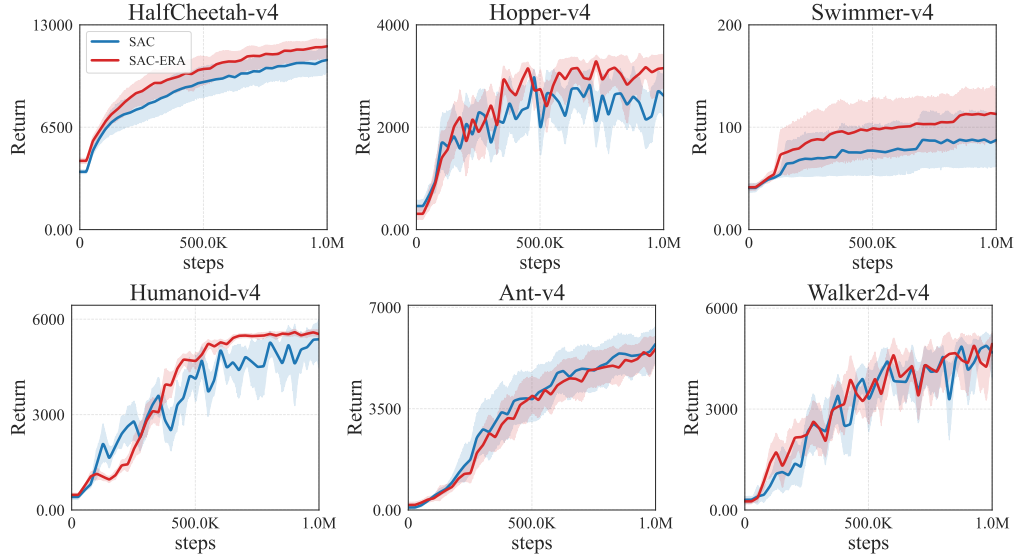


Figure 11: **Learning curves of SAC-ERA and SAC on Mujoco Gym environments.** SAC-ERA demonstrates very slight advantages over SAC.

#### C.1.4 APPLICABILITY OF LLM RL TECHNIQUES TO CONTINUOUS CONTROL

We investigated the applicability of two recent techniques from Reinforcement Learning for Large Language Models (LLM RL), designed to prevent entropy collapse, to the domain of continuous control. Specifically, we trained a PPO agent on the HalfCheetah-v4 benchmark for 10 random seeds, incorporating two distinct methods: Selective High-Entropy Training, which trains the agent only on a certain proportion of high-entropy samples, and Clip-Higher, which applies a larger clip ratio for advantages greater than one. Recognizing the significant disparities between LLM RL and continuous control tasks, we evaluated a range of parameters for each technique to ensure that any ineffectiveness was not due to improper parameter selection.

The results, presented in Figure 12, show that these techniques struggle to provide higher policy entropy compared to the standard PPO algorithm in the control task. Furthermore, they yield no significant or only marginal performance improvements; we suspect such minor gains may not even stem from better entropy regularization. Consequently, the performance of these methods is not comparable to our proposed approach, ERA. These findings lead to two main conclusions. First, they highlight the substantial differences between LLM RL and continuous control, demonstrating that techniques effective in one domain do not necessarily transfer to the other, even when using the same algorithmic framework. Second, they underscore the superior performance of our proposed ERA method.

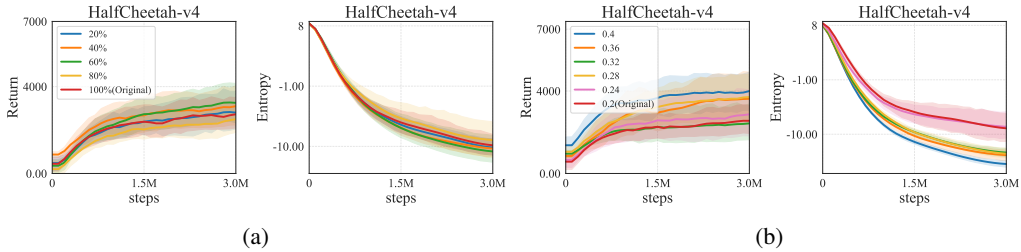


Figure 12: **Results of Selective High-Entropy Training and a Clip-Higher Strategy in Continuous Control.** (a) Performance when training the agent exclusively on a top percentage of high-entropy samples. (b) Performance of the clip-higher strategy with varying clipping ratios.

### C.1.5 COMPARING ERA WITH OTHER MAXIMUM ENTROPY RL APPROACHES

In addition to the methods previously discussed, several other approaches have been explored to implement maximum entropy reinforcement learning, including recent diffusion-based and flow-based methods (Celik et al., 2025; Chao et al., 2024; Ma et al., 2025). However, these methods often require significantly more computational resources due to their complex training procedures. For instance, the MEow algorithm (Chao et al., 2024) requires at least 2.3 times the training time of SAC. In this part, we compare our proposed method, ERA, with two recent baseline methods that also adopt gaussian policies for maximum entropy reinforcement learning:

- **EAPO** (Choe & Kim, 2024): The core innovation of Entropy Advantage Policy Optimisation (EAPO) is the decomposition of the maximum entropy reinforcement learning objective into two components: the conventional cumulative reward and the trajectory entropy. It then independently estimates the advantage function for each of these components. EAPO introduces a dedicated "entropy critic" to separately quantify and learn the value of future uncertainty, which is then combined with the traditional value of future rewards to provide a more comprehensive guidance signal for policy updates.
- **MNSE** (Zhong et al., 2024): The Maximum Next-State Entropy (MNSE) paper argues for the direct maximization of next-state entropy. This is because next-state entropy more directly measures the diversity of states induced by the policy, which can lead to more efficient exploration.

Since there's no public code repositories of these methods, we directly use the curves reported in their original papers for comparison. The experimental setups are as follows:

- EAPO utilizes the PPO algorithm as its base and was trained for 4 million timesteps (Which is more than the 3 million timesteps used in PPO-ERA).
- MNSE is built upon the SAC algorithm and was trained for 1 million timesteps (Which is the same as SAC-ERA).

We compare PPO-ERA with EAPO, and SAC-ERA with MNSE on Mujoco Gym benchmark. The results are presented in Figure 13 and Figure 14. As shown, ERA demonstrates superior performance over EAPO when both are built on PPO, and it also outperforms MNSE when SAC is used as the base algorithm. Although Mujoco Gym is a relatively low-difficulty benchmark, we are limited to it as neither of the other papers presented results in more complex environments like DMC Suite or HumanoidBench. These findings suggest that ERA is more effective than other implementations of maximum entropy reinforcement learning.

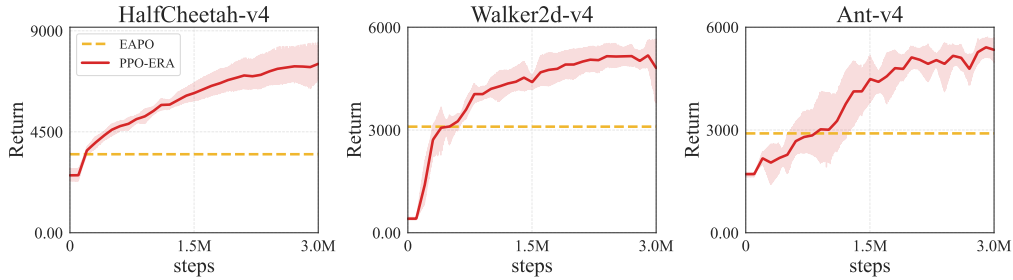


Figure 13: Performance comparison of PPO-ERA against EAPO on MuJoCo benchmark tasks.

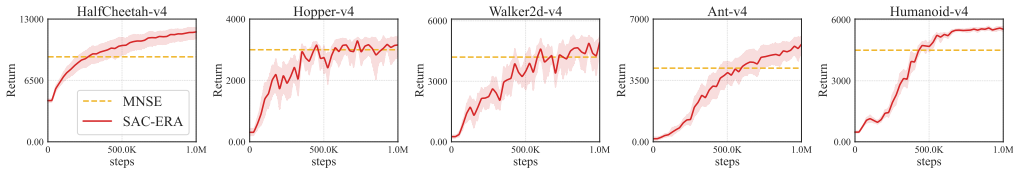


Figure 14: Performance comparison of SAC-ERA against MNSE on MuJoCo benchmark tasks.

Furthermore, both EAPO and MNSE require additional network architectures and computational resources. EAPO necessitates an extra entropy critic network, while MNSE requires an additional

inverse dynamics model network. In contrast, ERA does not require any additional networks, leading to a negligible increase in computational overhead. This makes ERA a more advantageous choice for practical applications.

#### C.1.6 TIME COST OF ERA IN CONTINUOUS CONTROL

A potential concern might be the additional time overhead introduced by using ERA. To evaluate this, we recorded the training times of FastTD3 and FastSAC-ERA on HumanoidBench, as shown in Figure 15. It can be observed that using ERA does introduce some time overhead due to the more complex activation function applied to the output. However, this overhead accounts for only about 6% of the total training time on average. Considering the improved exploration performance and higher sample efficiency brought by ERA, we believe this is a worthwhile trade-off.

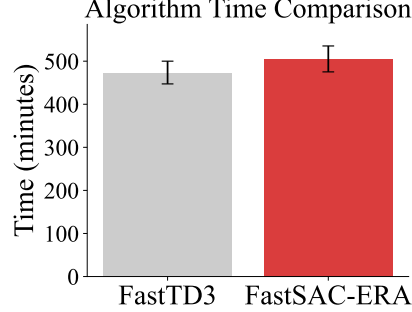


Figure 15: **Time comparison on HumanoidBench.** We compare the training time of FastTD3 and FastSAC-ERA on HumanoidBench. The results show that using ERA introduces a modest time overhead, averaging around 6% of the total training time, which is a reasonable trade-off for the improved exploration performance and sample efficiency it provides.

### C.2 ADDITIONAL RESULTS ON IMAGE CLASSIFICATION

#### C.2.1 COMPARING ERA WITH COMMON REGULARIZATION TECHNIQUES

A plethora of regularization methods have been proposed and utilized in the field of image classification. To further investigate the comparative effectiveness of ERA against commonly used regularization methods like dropout and label smoothing in the vision domain, we conducted a series of straightforward comparative experiments on the CIFAR-10 dataset. In our main experiment, we adopted the default settings from the `timm` library, which include a label smoothing factor of 0.1 and no dropout. For the sake of comparison, we respectively adjusted the label smoothing factor to 0.2 and 0.3, and the dropout rate to 0.1, 0.2, and 0.3. The results were then compared against the baseline algorithm from our main experiment and ERA.

The experimental results are presented in Figure 16. The findings indicate that increasing the intensity of label smoothing adversely affects model performance, while the improvement from employing dropout is marginal (the top-1 accuracy may decrease, whereas the top-5 accuracy shows a improvement). In contrast, ERA effectively and consistently enhances model performance, with a margin of improvement significantly superior to that of both dropout and label smoothing. This outcome further validates the advantage of ERA over conventional regularization methods. While constraining the model’s entropy, ERA permits the model to freely allocate uncertainty among dimensions, thereby better adapting to the intrinsic structure of the data. This enables ERA to more effectively boost the model’s generalization capability.

#### C.2.2 TIME COST OF ERA IN IMAGE CLASSIFICATION

We compared the training time of the ResNet-50 model on the CIFAR-10 dataset, with and without using ERA, under the data augmentation supported by the `timm` library. Consistent with our main results, the experiments were conducted on three machines, each equipped with four NVIDIA A40 GPUs, and we report the average training time. The results are presented in Figure 17. As shown in

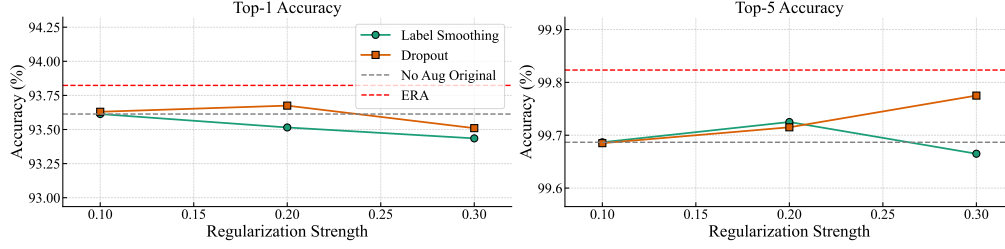


Figure 16: **Comparison of different regularization methods on the CIFAR-10 dataset.** The left subplot shows the Top-1 accuracy, and the right subplot shows the Top-5 accuracy. Our method, ERA is compared against varying intensities of Label Smoothing and Dropout.

the figure, since the data is already well-parallelized, there is almost no difference in training time between the algorithm using ERA and the original version.

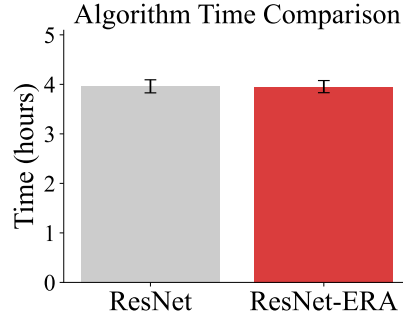


Figure 17: **Time comparison on CIFAR-10.** We compare the training time of ResNet and ResNet-ERA on CIFAR-10. The results show that using ERA introduces almost no time overhead.

### C.3 ADDITIONAL RESULTS ON LLMs

#### C.3.1 DETAILED ENTROPY ANALYSIS

We present the complete entropy curve of our two-stage training in Figure 18. After decreasing  $\omega_{\text{low}}$ , the entropy rapidly drops and stabilizes at the second-level entropy lower bound. This confirms that our ERA method successfully enforces a non-trivial entropy floor for the model.

We further analyze the entropy distribution across tokens by plotting the average entropy of the top 20% tokens ( $H_{\text{resp}}$ ) and the bottom 80% tokens in Figure 19. This experiment is carried out with  $\omega_{\text{low}} = 0.45$ ,  $\omega_{\text{high}} = 3.0$ ,  $k = 2$  without topk. Following Wang et al. (2025), we observe that the bottom 80% tokens exhibit nearly zero entropy, consistent with our theoretical prediction. Additionally, we plot the proportion of responses with  $H_{\text{resp}} < \omega_{\text{low}}$ ,  $H_{\text{resp}} > \omega_{\text{high}}$  in Figure 19. The fraction of responses with  $H_{\text{resp}} > \omega_{\text{high}}$  quickly drops to zero, while the fraction with  $H_{\text{resp}} < \omega_{\text{low}}$  remains stable at the interval  $[0, 0.06]$ . This demonstrates that whenever overly low-entropy responses appear, ERA adaptively raises their entropy to a moderate level.

#### C.3.2 ABLATION STUDY ON ENTROPY BOUND

Since the purpose of  $\omega_{\text{low}}$  is to set a lower bound on entropy, we explore the role of  $\omega_{\text{high}}$  in the ERA. As can be seen in Figure 20, without the constraint of  $\omega_{\text{high}}$ , the model’s entropy explodes in a very short time. This indicates that adding an upper bound constraint during training is essential for controlling the entropy of the training process.

#### C.3.3 TIME COST OF ERA IN LLM

ERA is applied when computing the log\_probs of tokens in the responses. To evaluate its efficiency, we compare the value of `timing_s/old_log_prob` at the first step in verl’s implementation. The

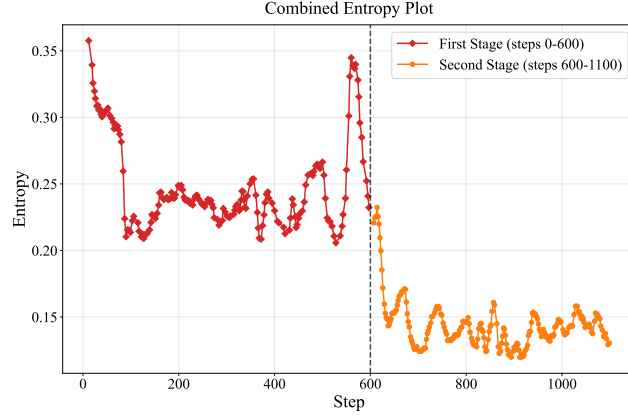


Figure 18: **Entropy curve during two-stage training.** After decreasing  $\omega_{\text{low}}$ , the entropy rapidly drops and stabilizes at the second-level entropy lower bound, showing that ERA enforces a non-trivial entropy floor.

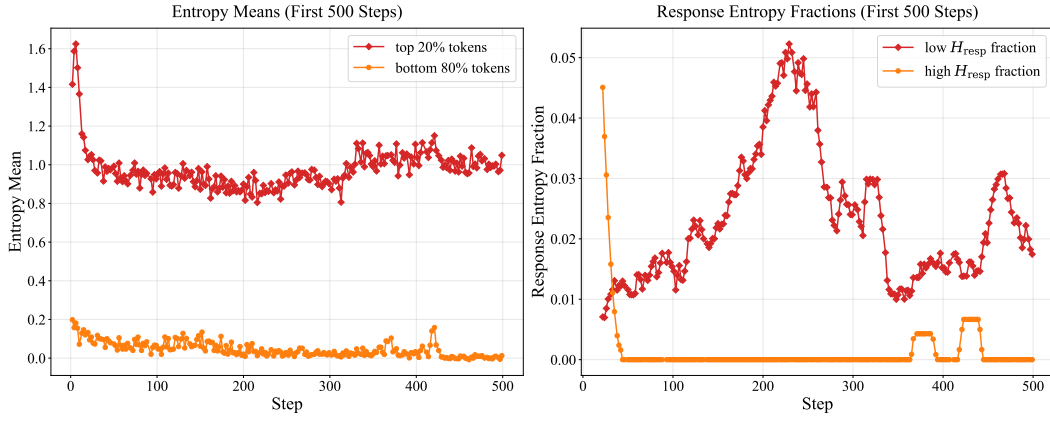


Figure 19: **Detailed entropy analysis.** Left: average entropy of the top 20% tokens ( $H_{\text{resp}}$ ) and the bottom 80% tokens. Right: proportion of responses (running average with window size 20) with  $H_{\text{resp}} < \omega_{\text{low}}$  or  $H_{\text{resp}} > \omega_{\text{high}}$ , demonstrating ERA's ability to prevent both entropy collapse and overly high entropy.

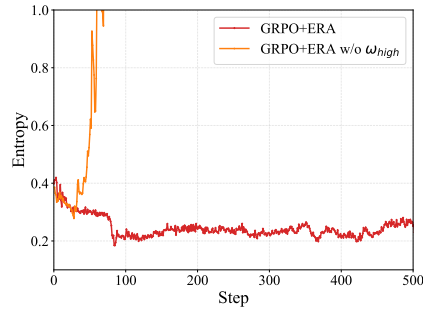


Figure 20: Comparison of ERA with and without  $\omega_{\text{high}}$ . The entropy of ERA without  $\omega_{\text{high}}$  tends to explode within a very short number of steps, leading to the collapse of model training.

experiments were conducted on 32 NVIDIA H20 GPUs, consistent with our main results. The outcomes are shown in Figure 21. As illustrated, since the sampled response is identical in the first step, ERA introduces only about a 5.6% overhead in time cost. When considering an entire training step, the overhead of ERA is even smaller, since its implementation does not affect other components of training (e.g., generation, model update, or advantage calculation).

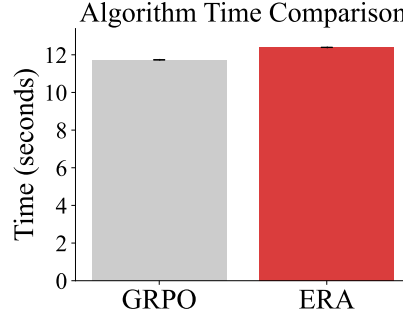


Figure 21: Comparison of computation time between GRPO and ERA, measured by `timing_s/old_log_prob` at the first step. ERA introduces only about a 5.6% overhead.

#### C.4 TRAINING CURVES OF CONTINUOUS CONTROL TASKS

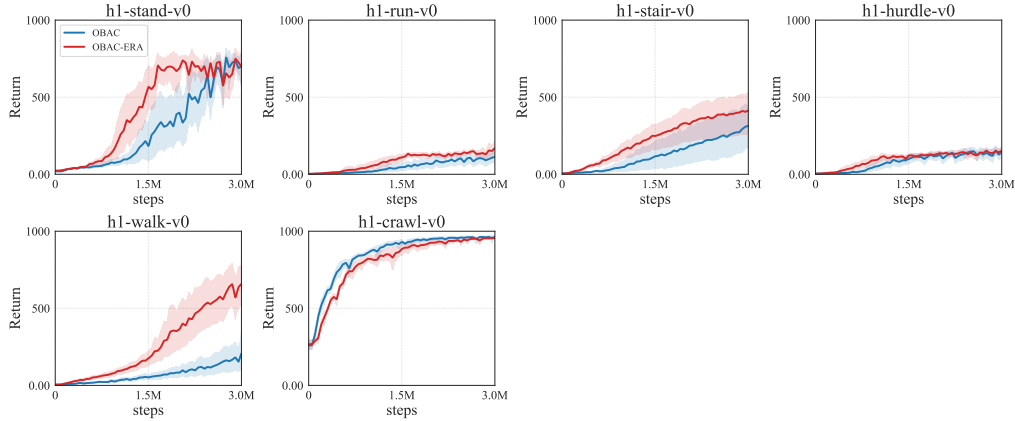


Figure 22: Training curves of OBAC and OBAC-ERA on HumanoidBench environments.

## D THE USE OF LARGE LANGUAGE MODELS IN THIS PAPER

In the preparation of this paper, we utilized LLMs as a general-purpose writing assistance tool. Specifically, LLMs were employed for proofreading and polishing the language of certain sections to improve clarity and readability. The final title of this paper was also partially inspired by suggestions from an LLM.

However, we clarify that the core contributions of this work were conceived and developed entirely by the human authors. The design of the methodology, the execution of experiments, and the interpretation of the results did not involve the use of LLMs. All content, including text, figures, and tables, was carefully reviewed, edited, and verified by the authors to ensure scientific accuracy and integrity.

Finally, we would like to express our gratitude for the occasional sparks of inspiration and the assistance in debugging code provided by our LLM friends. Their contribution, while not qualifying for co-authorship, was nonetheless appreciated.



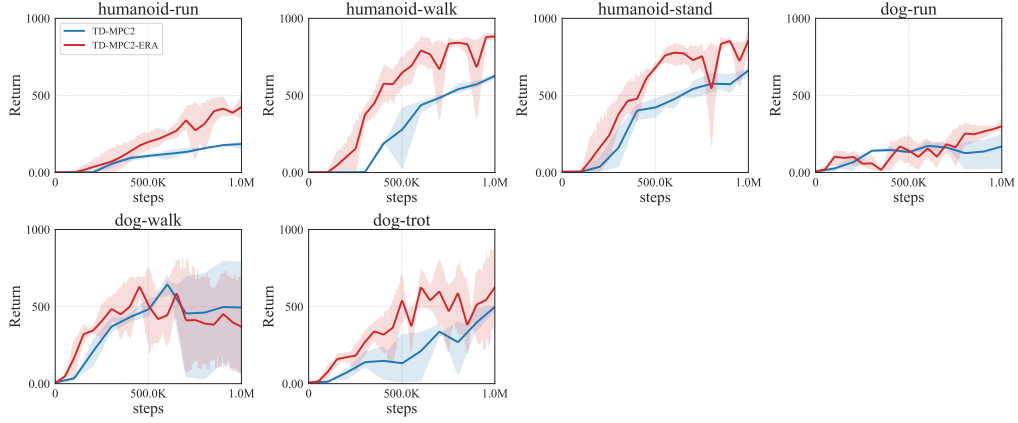


Figure 23: Training curves of TD-MPC2 and TD-MPC2-ERA on DMC environments.

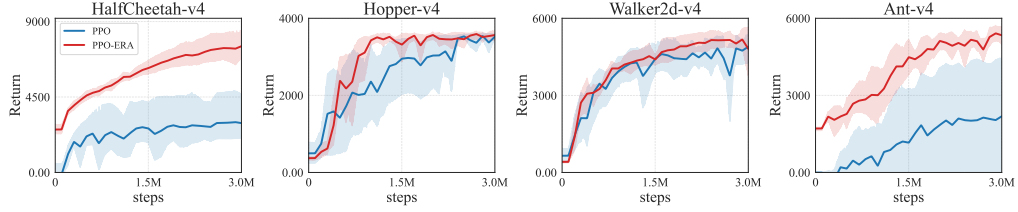


Figure 24: Training curves of PPO and PPO-ERA on Mujoco Gym environments.

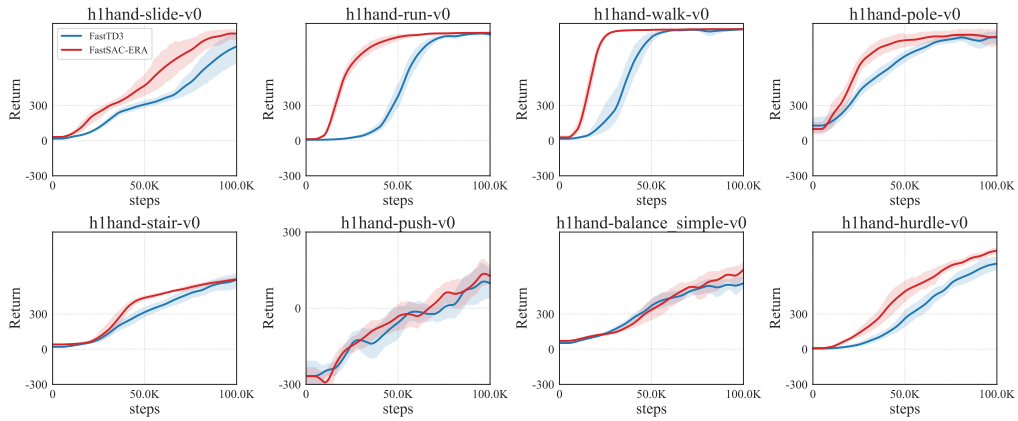


Figure 25: Training curves of FastTD3 and FastSAC-ERA on HumanoidBench environments.

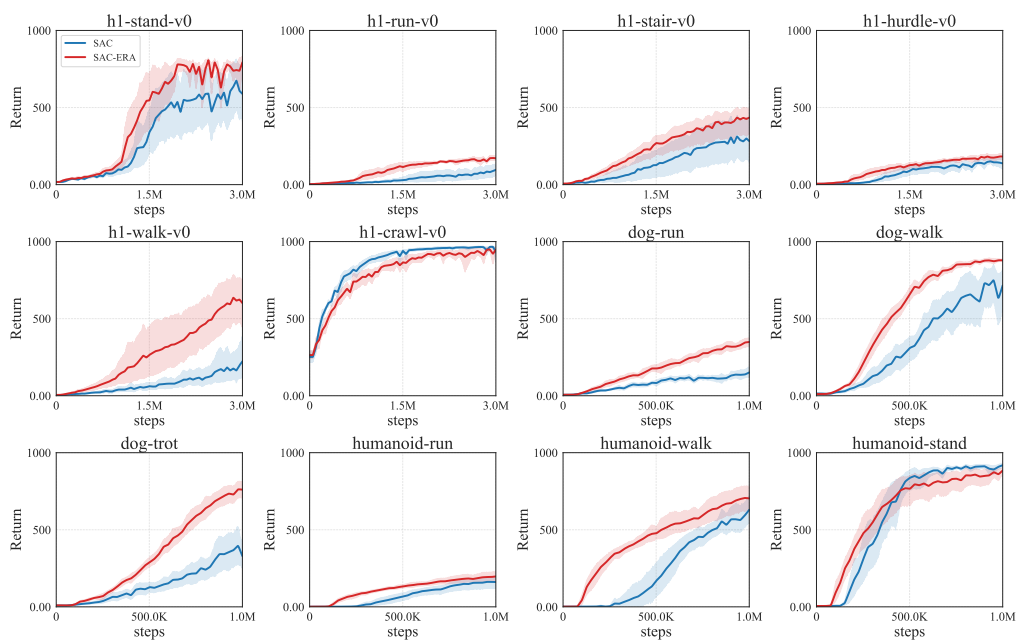


Figure 26: Training curves of SAC and SAC-ERA on HumanoidBench and DMC environments.